



Ayezabu Amanuel

Supabase vs Firebase: Evaluation of performance and development of Progressive Web Apps

Metropolia University of Applied Sciences

Bachelor of Engineering

Information Technology

Bachelor's Thesis

20 May 2022

Abstract

Author:	Ayezabu Amanuel
Title:	Supabase vs Firebase: Evaluation of performance and development of Progressive Web Apps
Number of Pages:	58
Date:	20 May 2022
Degree:	Bachelor of Engineering
Degree Programme:	Information Technology
Professional Major:	Mobile Solutions
Supervisors:	Patrick Ausderau, Senior Lecturer Oleksii Razbakov, Senior Developer

The purpose of the thesis is to evaluate Firebase, and Supabase which are platforms that provide developers with products and services with the goal of helping to build applications. The platforms were compared under different criteria such as features, performance, products, prices, and stability in order to identify the advantages and disadvantages of each platform. The evaluation was done for WeDance, a non-profit social network for dancers. It was done with the aim of providing WeDance the information required to make the decision to switch between the two platforms based on the necessities of the company.

The research was carried out by reviewing existing literature to gain proper insight on the topic. Additionally, two identical progressive web applications were built using Firebase and Supabase. The main functionality of the application was to send users daily weather reports for locations users had specified. The goal was achieved by using different back-end services such as authentication, social integration, storage, and scheduled jobs.

The results reveal that selecting a more suitable platform between the two is dependent on the use case of the project. For instance, Supabase offers a relational database, as well as full-text search capabilities which can be great for projects that make lots of queries. Firebase on the other hand offers a NoSQL database with offline support which can be great for building progressive web applications.

Furthermore, the performance test results acquired after examining the applications indicated that the application built with Supabase had a faster load speed on average. Additionally, the Supabase application was also able to sustain more consecutive requests. However, the Firebase database has a

higher load threshold, but it can only be reached by incrementally increasing the request over time.

The results obtained can be used by developers, and businesses to make informed decisions when picking one of these two platforms for a project. Lastly, the research could be further expanded by evaluating other aspects of Firebase and Supabase that are not thoroughly covered in this paper. For instance, the self-hosting feature of Supabase or the database scalability of Supabase.

Keywords: Firebase, Supabase, Progressive Web App, Backend as a Service

Contents

1	Introduction	1
2	Research Design	2
2.1	Research Questions	2
2.3	Research Methods	3
3	Modern Web Development Approaches	4
3.1	Front-End Development	5
3.2	Back-End Development	8
3.3	Next.js and React	8
3.4	Backend-as-a-Service	12
3.4.1	Advantages of BaaS	13
3.4.2	Disadvantages of BaaS	14
3.5	Progressive Web Apps (PWAs)	15
3.6	Fundamental criteria of PWA	17
4	Firebase	20
4.1	Realtime Database and Cloud Firestore	20
4.1.1	Realtime Database	20
4.1.2	Cloud Firestore	21
4.2	The Difference between Cloud Firestore and Realtime Database	22
4.3	Authentication, Cloud and Hosting	25
4.3.1	Authentication	25
4.3.2	Cloud Storage	26
4.3.3	Cloud Functions	27
4.3.4	Hosting	27
5	Supabase	28
5.1	Database and Authentication	29
5.2	Supabase Storage and Edge Functions	30
6	Implementation	32
6.1	The case project	32

6.1.1	Case project description	32
6.1.2	Project environment and file structure	34
6.1.3	Implementing the UI	35
6.2	Firebase implementation	36
6.2.1	Getting started with Firebase	36
6.2.2	Implementing authentication and user management with Firebase	38
6.2.3	Implementing the main functionality with Firebase	40
6.3	Supabase implementation	42
6.3.1	Getting started with Supabase	42
6.3.2	Implementing authentication and user management with Supabase	43
6.3.3	Implementing the main functionality with Supabase	44
6.4	Creating a PWA with NextJS	45
7	Results	47
7.1	Advantages of Firebase over Supabase based on different aspects	47
7.1.1	Firebase features	47
7.1.2	Firebase stability	48
7.1.3	Firebase database	48
7.1.4	Firebase functions	49
7.1.5	Firebase hosting	49
7.1.6	Firebase PWA features	50
7.1.7	Firebase pricing	50
7.2	Advantages of Supabase over Firebase based on different aspects	52
7.2.1	Supabase features	52
7.2.2	Supabase database	52
7.2.3	Supabase Pricing	53
7.3	Performance Tests	53
8	Conclusion	57
	References	58

List of Abbreviations

API:	Application Programming Interface. It is a set of functions, commands, and objects used to communicate with software.
BaaS:	Backend-as-a-Service. It is a cloud computing service model that allows developers to link their applications to backend cloud services via application programming interfaces and software developers' kit.
CAGR:	Compound Annual Growth Rate. It is the mean annual growth rate of an investment over a specified period longer than one year.
CLI:	Command Line Interface. It is used to operate software and operating systems via commands.
CRUD:	Create, Read, Update, and Delete. They are the four basic operations performed on stored data.
CSV:	Comma-Separated Values. It is a way to store structured data in plain text format.
E-Commerce:	Electronic Commerce. It refers to business over the internet.
GPS:	Global Positioning System. It is a satellite navigation system used to determine the ground position of an object.
GraphQL:	Graph Query Language. It is a language used to define how a client should request data using an application programming interface.
HTTP:	Hypertext Transfer Protocol. It is the protocol used to transfer data over the internet.
HTTPS:	Hypertext Transfer Protocol Secure. It is the protocol used to securely transfer data over the internet using a secure socket layer.

iOS:	iPhone Operating System. It is a mobile operating system developed by Apple.
JSON:	JavaScript Object Notation. It is a text-based data interchange format used for transmitting structured data.
JSX:	JavaScript XML. It is a syntax extension to JavaScript used with React to describe what the UI should look like.
MVC:	Model-View-Controller. It is a software design model.
MVP:	Minimum Viable Product. It is a product with enough features to be released to early consumers.
NoSQL:	Not only SQL. It is a non-relational database that uses key-values to access and store data.
NPM:	Node Package Manager. It is a package manager for JavaScript's runtime Node.js
PWA:	Progressive Web App. A browser-based application that also has native-like features.
SDK:	Software Development Kit. It is a collection of software tools used for developing applications for a specific operating system or device.
SEO:	Search Engine Optimization. A set of practices designed to improve the visibility of a website for search engines.
SSG:	Static Site Generator. It is a tool that generates static HTML pages from templates and raw data.
SSL:	Secure Sockets Layer. It is a security protocol developed for transporting data over the internet.

- SSR: Service-Side Rendering. It refers to an application's ability to fully render HTML pages on the server instead of the client.
- SQL: Structured Query Language. It is a query language used for accessing and modifying data in a database.
- UI: User Interface. The way through which a person can interact with a software application or a hardware device.
- URL: Uniform Resource Locator. It is the address of a specific web page or file on the internet

1 Introduction

In today's age, the web is an essential part of people's lives. With the growth of technology, web development has become increasingly advanced.

Consequently, writing software for the web has become more sophisticated. To alleviate the challenges that arise when writing software for the web, there exists a plethora of software tools that developers can utilize to help simplify and speed up the process of building a web application. The term software tool encompasses a large spectrum of development tools covering anything from text editors to software testing tools. This thesis is mainly going to be focusing on application development tools or services that help to produce web applications more efficiently, in particular Firebase and Supabase.

Firebase and Supabase provide a variety of backend features such as authentication and database that enable developers to build Android, iPhone Operating System (iOS), and Web Application. Although the services are provided for different platforms, this thesis focuses on the services provided for web development. As part of the development team at WeDance, a social network for connecting dancers and non-profit international dance communities [1], the designated task was to evaluate Firebase and Supabase. Currently, Firebase is the tool being utilized by WeDance as it is a very well-established development tool. However, there exists a huge interest in considering Supabase as an alternative to Firebase. The objective of this final year project is to examine and compare the two, to provide WeDance with the necessary information, before deciding which service to use.

Which development tool is the most viable option for WeDance? To help answer this question, an application with two different backends but similar features was built using both Firebase and Supabase. The frontend was then implemented using Next.js, a development framework built on Node.js. As part of the development process, the examination was carried out based on products, stability, features, performance, and pricing of the services. Additionally, both tools were further examined from the perspective of Progressive Web Apps (PWAs), to decide which platform offered better services

for building an application with progressive features. Finally, a comparison between the two was made to help identify the strengths and weaknesses of each development tool.

Overall, this paper contains four sections. Following the introduction, chapter 2, 3, 4, 5 discusses modern web development approaches, PWAs, and Backend-as-a-Service (BaaS). Chapters 6 and 7 focus on comparing Firebase and Supabase as part of the development process. Lastly, chapter 8 concludes the thesis.

2 Research Design

This section lays out the research design or blueprint for the case study. It helps identify the research questions, and the overall objectives. In addition, it clarifies the approach or method used for conducting the research and acquiring data. In doing so, it aims to ensure that the results obtained can be effectively attributed to the research problem without any ambiguity [2].

2.1 Research Questions

Before defining the objectives, the main research question was deconstructed into sub-questions to gain a better understanding of the question or problem.

The research question in hand: *Supabase vs Firebase: Evaluation of performance and development of Progressive Web Apps.*

The list of sub-questions formed from the main question are:

- What are the advantages or disadvantages of either one over the other?
- Which of the two offers better PWA support?

The sub-questions break down the main question into two different parts to help clarify what exactly is being asked as the project question, and to help identify the specific strategy needed to arrive at a conclusion for the question in hand.

2.3 Research Methods

The main research question demands the comparison of two different software service providers, with the aim of discovering new information or arriving at a conclusion, therefore comparative research was carried out. The logical approach used was to deductively arrive to a specific conclusion from existing theory.

Using existing knowledge in the form of articles and journals, a theoretical background on the topic was formed. The theoretical background uses writings from reliable sources, which is properly reviewed, compared, and summarized. The writing was formed in accordance with Marco Pautasso's guideline on literature reviews [3].

In addition to analyzing existing papers, as a case study, two identical applications were built with the same functionality using two different services, Firebase and Supabase. They were both examined by performing the same task on both applications, and any differences in results were noted. Moreover, the research uses both quantitative, and qualitative research to analyze data. The former is used to understand general facts about a topic and deals with numbers and statistics, while the latter is used to understand ideas, and deals with words and meanings. [4]

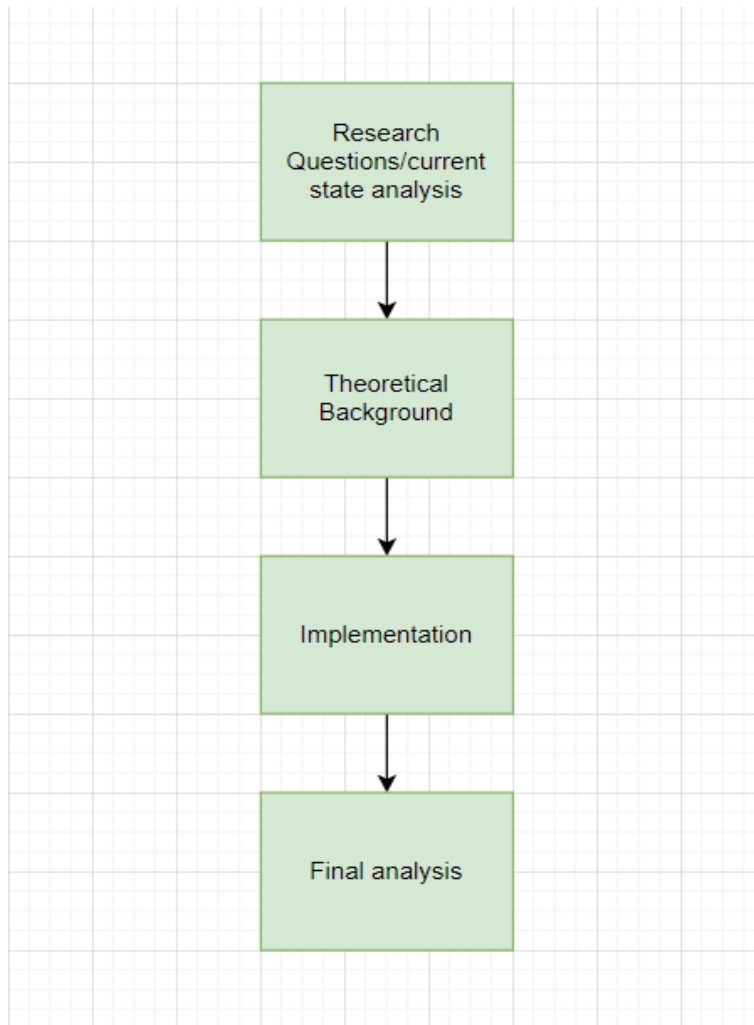


Figure 1. Research structure

The general steps taken to conduct the research for the project is outlined in figure 1. The overall process is composed of four major parts. As part of the research process, information relating to the current state of the topic was also analyzed. The tools currently being used were highlighted, and the weaknesses identified with the current system by the founder of the case company, WeDance, were gathered.

3 Modern Web Development Approaches

This chapter covers some of the fundamentals of web development. It also explains some of the concepts and common approaches used in web

development. It aims to highlight the different methods and tools used to achieve modern web experiences. Furthermore, it explains the reasoning behind the preferred selection of tools and methods for this project.

3.1 Front-End Development

Web applications generally consist of a frontend and a backend. They both play a crucial role in forming a fully functional web application. However, they each play a different role and are easily distinguishable. The word frontend is a commonly used term amongst developers and is interchangeable with client-side. The term is used to describe everything a user can visualize and interact with. For instance, when visiting an E-commerce (electronic commerce) site such as Amazon, the list of items that users can interact with by browsing and adding in shopping carts is handled in the frontend. In Fact, it may seem as if everything is handled in the frontend, but most of the workload is generally managed in the backend. [5]

It is important to note that, although the backend is very crucial, it is not always necessary to have a backend when developing a web application. The necessity of a backend is dependent on the use case of the application. In certain cases, it might not be a requirement for an application to fetch or store data. For example, a simple portfolio site that serves static or fixed content can be built mainly using frontend technology, although a cloud service would still be required for hosting static files.[5]

A few central technology stacks utilized in frontend web development comprise Hypertext Markup Language (HTML), Cascading Style Sheet (CSS), and JavaScript. HTML is a markup language that is responsible for creating and organizing the content on a website [6]. HTML is an easily readable language that uses tags to define elements inside a document [7]. CSS is a language that is used for styling and formatting a website's content, such as color, layout, and font [6].

To make websites livelier, a programming language such as JavaScript is used to make web pages interactive and dynamic [7]. It is a powerful language that is interpreted at runtime, when a program is running, by the web browser. While it is well recognized for its client-side use, it is also used in a great deal of non-browser environments, such as Node.js, Apache CouchDB, and Adobe Acrobat [8].

HTML, CSS and JavaScript are the basic building blocks used in frontend web development. When working with large frontend web applications, libraries or frameworks that provide additional support are generally used. Some of the most popular choices include React, Vue.js, and Angular [9]. All these frameworks provide a foundation or boilerplate on which developers can build applications that are scalable and easily maintainable [7].

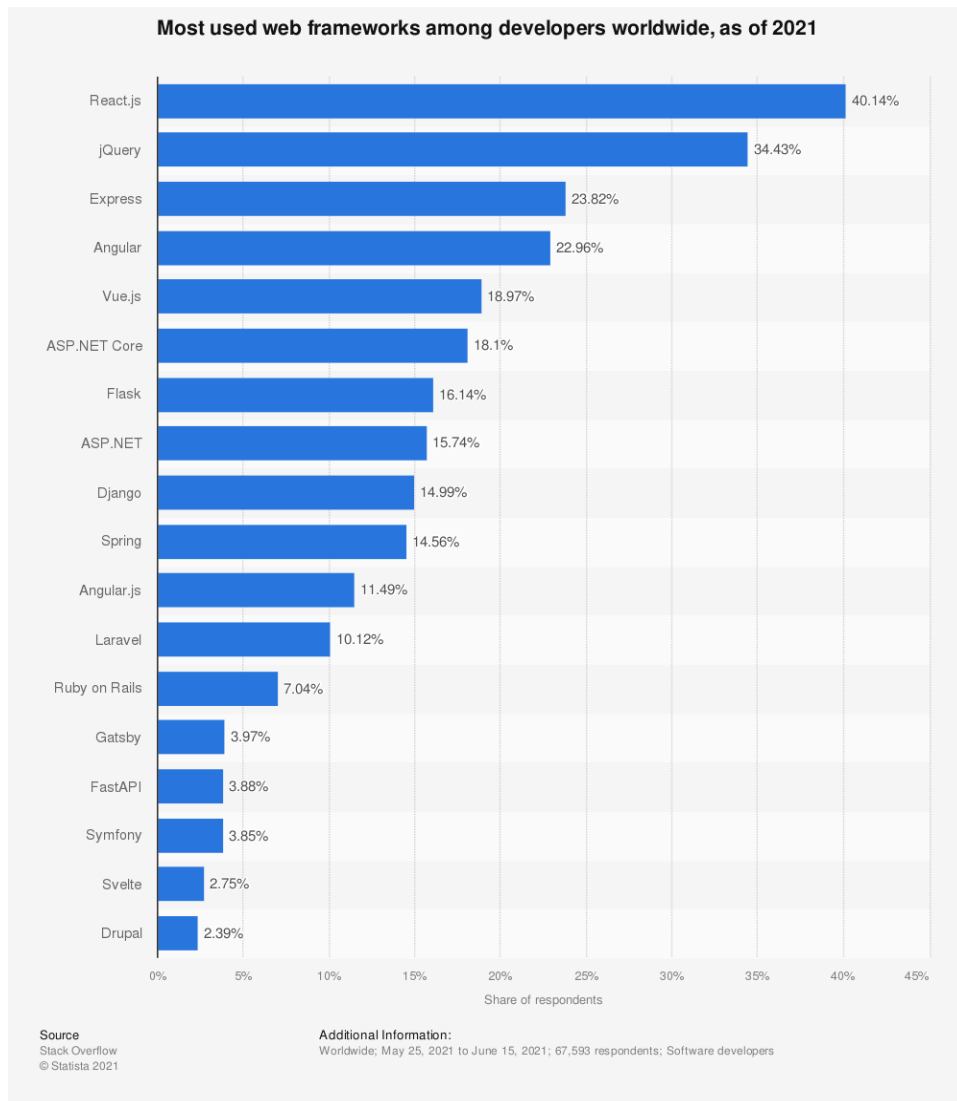


Figure 2. The most used web frameworks among developers worldwide, as of 2021. [9]

Figure 2 provides some interesting information about the most used web frameworks by developers, as of 2021. According to the survey, 40.41 percent of respondents use React, making it one of the most used libraries. Amongst the top JavaScript frameworks mentioned in the survey, Angular and Vue.js are ranked in fourth and fifth place respectively. In third place, Express is ranked at 23.82 percent making it the only backend framework in the top five.

3.2 Back-End Development

The backend refers to all the technology required to handle incoming requests from the client, process them, and then send a response back. The method in which the frontend communicates with the backend is via Hypertext Transfer Protocol (HTTP) requests. Typically, the backend consists of three major parts. The first major part is the server, which is the computer program or device that listens for incoming requests. The second major part is the app, which is the application program running on the server that receives requests, queries data from the database, and then sends a response to the client. Lastly, the remaining major part is the database, which is used to persist and organize data.[10]

In addition to the above-mentioned, the backend is also responsible for handling server-side processes. Some of which may include Encrypting and decrypting data to protect sensitive information, handling file uploads, long-lasting operations, interacting with the file system, and validating input, which can also be done client-side. [11]

When working on the backend, programming languages that apply to its development are used. Some popular choices include Node.js, Java, Python, Ruby on rails, and numerous more. Generally, since both the frontend and backend are required to deliver a full user experience, developers write code that is specific to both, also known as full-stack development [11].

3.3 Next.js and React

Next.js is an open source React framework built by Vercel and the open-source community. It offers the fundamental building blocks needed to develop web apps more efficiently. Building web applications involves the consideration of common application requirements, such as user interface (UI), routing, data fetching, rendering, integration, infrastructure, performance, scalability, and developer experience. [12]

A user interface is a broad term that describes human to computer interactions. This can include peripheral devices, or even a command line interface. However, in this case the word is used to describe the way through which a user interacts with an application.

Routing is the technology used to change between different UI views or how a user can navigate between different parts of an application. Data Fetching is where data lives and the different ways in which data is retrieved. This includes Client-side rendering, Static-site Generation, and pre-rendering data with Server-side Rendering. Rendering in this case refers to when or where static or dynamic content is presented to be interacted with.

Integration involves the consideration of how the application interacts with third-party services such as authentication or payment. Infrastructure is where application code is stored, deployed, and run. Performance looks at how the experience of the end-user can be improved. Scalability considers how an application will adjust as traffic and data increase overtime. Lastly, developer experience describes the experience of developer teams maintaining and dealing with a code base.

For each one of these development considerations, a solution is needed, and developers need to build it themselves. Oftentimes, it can be an arduous process, which is why tools such as libraries and frameworks exist. To understand what Next.js does, React must be first defined.

React is a JavaScript library used for building interactive and composable user interfaces. The library offers helpful JavaScript functions for building self-encapsulated component-based UI, with the ability to manage its own state. Component logic is written in JavaScript but React offers JavaScript extensible markup language (JSX), which makes writing markup that resembles HTML syntax possible inside React components, and Classes. [13]

Creating components in a React application allows the separation of UI into independent, and reusable chunks. Components are created using JavaScript, which makes handling data easier. Data mutation inside React components

causes a re-render without reloading the entire page. Moreover, data-flow architecture in React is unidirectional, meaning data is transferred from a parent component to a child component in the form of a prop, but not vice versa. This architecture design makes the data-flow clear. [14]

```
class HelloMessage extends React.Component {
  constructor(props) {
    super(props);
    this.state = { message: 'welcome' };
  }
  render() {
    return <div>Hello {this.props.name}, {this.state.message}</div>;
  }
}
root.render(<HelloMessage name="Taylor" />);
```

Listing 1. A stateful React component [13].

As seen in listing 1, data passed to a component can be accessed via `props`, and internal state data is accessible via the `state`. When a React component's state changes, it will cause the component to re-render by re-invoking the render method. [13]

React touches only on the view aspect of Model-View-Controller (MVC), but it does not focus on the other aspects of building web applications. For this reason, there exists an entire ecosystem of third-party tools that try to address the missing aspects by providing solutions. The upside of this is that there exists a plethora of third-party tools to pick from, but the downside is that it requires time and effort to configure tools and come up with different solutions for common application requirements.[13]

This is where Next.js comes into the picture. As a React framework, it handles the tooling and configurations needed for React. It introduces structure, optimization, and new features to React applications. Next.js eliminates all the tedious aspects of building a React app and improves the overall developer experience. It does this by providing some key building blocks that can be used to create web applications efficiently. [12]

For starters, Next.js does not require any configuration when setting up a new project since everything is preconfigured. This includes automatic code

compilation, minification, bundling, and many more. Minification is a process used to minimize code and markup in websites by removing redundant characters, code, whitespace, and comments. It helps to reduce the load time and bandwidth usage of websites. Bundling is another mechanism used to create code ready for production by merging multiple files into a single bundle file. This is especially useful when working on a large project that has a lot of dependencies.

When working with React from scratch, it is required to manually setup a bundler, a minifier, and a compiler using a JavaScript module bundler such as Webpack, and a JavaScript compiler such as Babel. This process can be very tedious and repetitive, hence why React toolchains such as Next.js, and Create React App exist. Although Create React App also offers a simple setup, there are a few distinctions between Create React App, and Next.js. For instance, Create React App requires a third-party library for client-side routing whereas Next.js includes its own filesystem based routing where components inside the `pages` directory become a route. Additionally, Next.js supports Application Programming Interface (API) routes where any file inside `pages/api` folder is treated as an API endpoint to provide backend functionality.

The major difference between Next.js, and other React toolchains is how content is rendered. Next.js offers Hybrid Static Site Generation (SSG), Server-Side Rendering (SSR), and Client-Side Rendering (CSR). With SSG, content of the site is generated at build time or when the project is compiled. With SSR, the server renders the page at request time and responds to the client with the content ready. Lastly, with CSR the content of the website is rendered in the browser with javascript. Create React App only offers CSR by default.

In addition to providing different rendering methods, Next.js also offers fast refresh, image optimization, code splitting, typescript support, and plenty more additional features but the ones already mentioned are adequate for it to be the preferred selection of framework for the project.

3.4 Backend-as-a-Service

Backend-as-a-service (BaaS) is a cloud service model that automates server-side development which allows developers to outsource server-side responsibilities to a third-party platform and focus on client-side development [15]. It helps to facilitate the development of mobile or web applications by connecting the front-end with the back end via application programming interfaces (APIs) and software development kits (SDKs). Generally, BaaS vendors provide several ready to use software tools for back-end development, such as data management, authentication, cloud storage, cloud functions, and social media integrations [15]. Plenty of vendors provide BaaS, some of which include Amazon Web Services (AWS) Amplify, Firebase, Supbase, and Back4App.

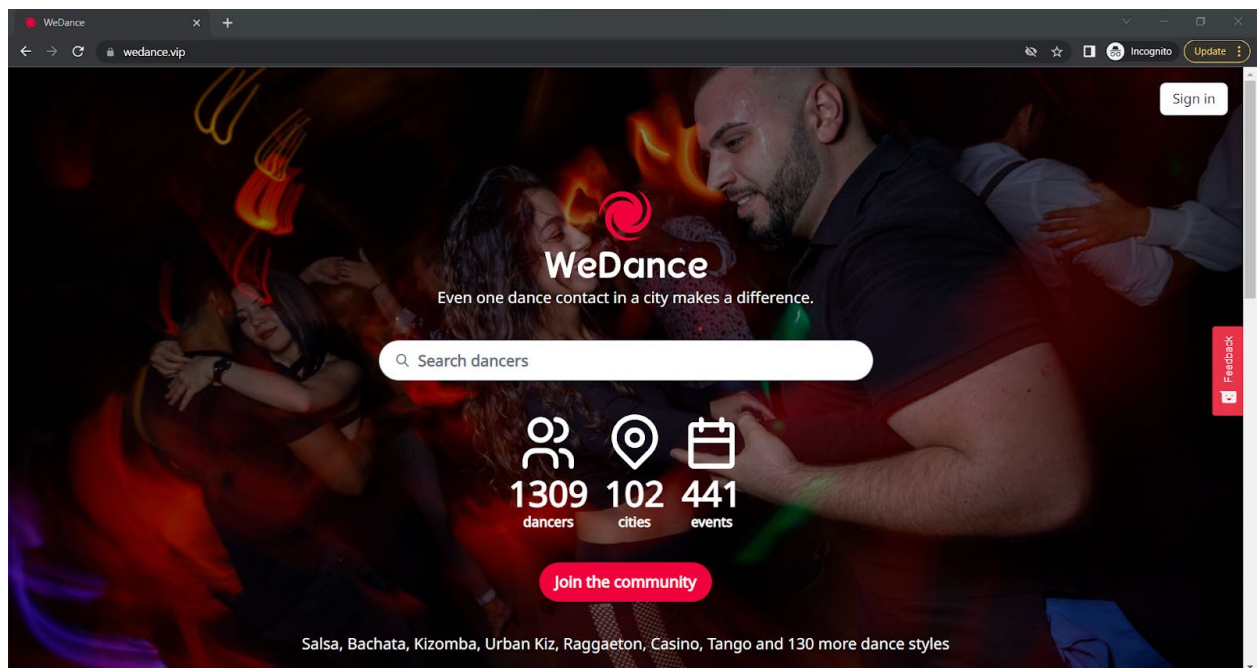


Figure 3. The homepage for WeDance.

In the case of WeDance, the website showcased in figure 3 is powered by Firebase. The platform is feature-rich, which is why Firebase was the preferred selection of tool used to build the backend of the website. Despite the great features that the platform provides for backend development, there are a few limitations that have caused the consideration for an alternative platform, such

as Supabase. The case company is mainly concerned with limitations faced with the Firebase database capability to perform certain tasks, such as compound search query.

3.4.1 Advantages of BaaS

According to the industryARC, the Cloud/Mobile Backend as a service market is forecast to reach \$7.2 billion by 2026, growing at a compound annual growth rate (CAGR) of 22.9% from 2021 to 2026 [16]. The growth in popularity of BaaS is due to various reasons. Some major ones include development speed, reducing time to market and costs.

The most obvious advantage of using BaaS solutions is the increased speed of development. BaaS solutions provide reusable code that can help reduce time wasted on building repeatable tasks. Software development is costly, and time consuming, and since most development tasks are handled manually, BaaS provides the ability to automate time consuming tasks. This can lead to an increased growth in productivity of software development [17].

A significant reduction of time to market is a direct consequence of increasing the development speed [17]. According to Ask Your Target Market (AYTM), an online research company, research involving one hundred developers was conducted. The results revealed it would take a total of eighteen weeks to build a minimum viable product (MVP)-quality native product. Ten of those weeks were spent solely on backend development. Taking these values into consideration, it is safe to say that considering BaaS as a solution can significantly reduce the total time of production, and therefore reducing the time to market [18].

According to Indeed, the average yearly salary of a software developer in Finland is \$49,203.

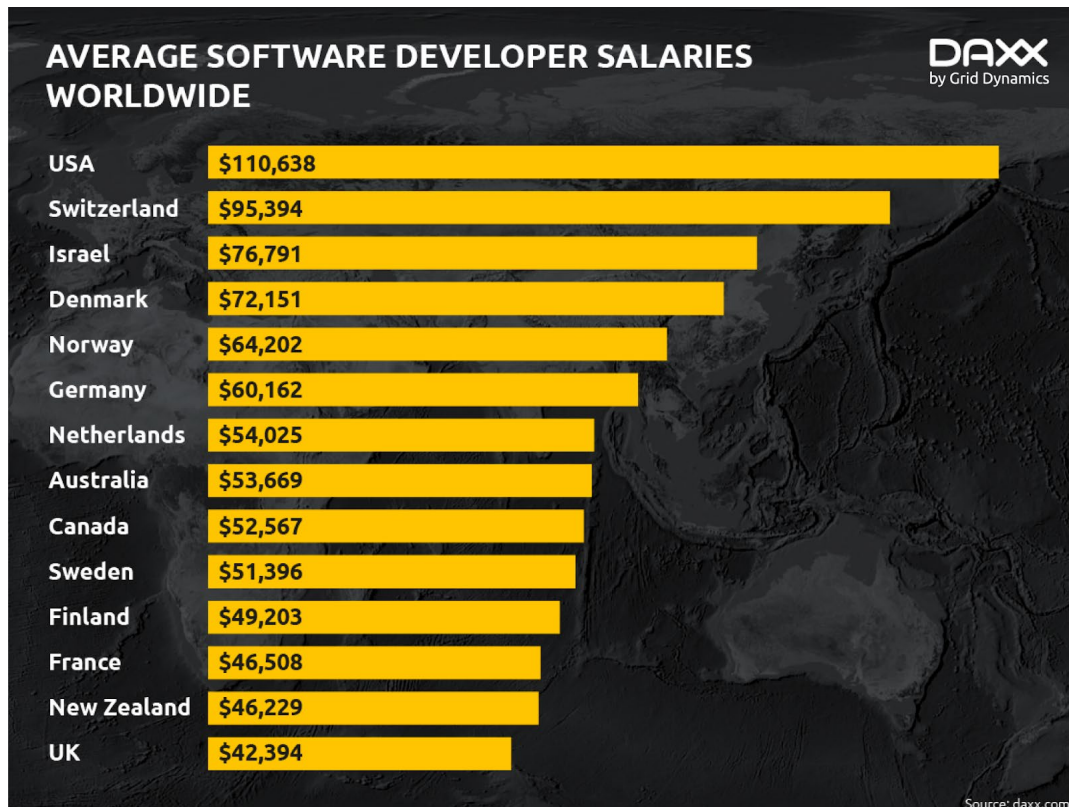


Figure 4. The average software developer salary around the world (2021). [19]

As illustrated in figure 4, hiring developers can be costly. With this in consideration it can be said that allocating developers time to work on low priority or repetitive tasks such as setting up a database or authentication is a waste of resources. BaaS solves this issue, as it plays an important role in helping automate repetitive tasks needed to build a backend. This eliminates the need for a larger developer team. Simultaneously, the developers that are needed can devote their time working on key features that define a product. [18]

3.4.2 Disadvantages of BaaS

Although BaaS can be time-effective and less costly, in certain use cases it is not the best solution. Especially when building an application that plans to grow fast, is expected to have a large user base, requires a unique or custom functionality that is not provided by any BaaS or is expected to be on the market for a prolonged period. In these cases, the preferred solution should be a

custom backend. The reason behind this conclusion is explained by a few disadvantages. [18]

BaaS platforms aim to satisfy the general requirements of most applications such as create, read, update, delete (CRUD) operations or social media integrations. However, some software applications need unique functionality to stand out. Because it is highly likely that it is not provided by a BaaS platform, building a custom backend is preferable as it provides more room for flexibility. [18]

BaaS is a competitive market where prices can change abruptly, and the increase in cost because of growth in traffic is not easily negotiable. Despite platforms such as Firebase providing a no-cost plan, for high level use cases, it is a requirement to switch to a paid-tier pricing plan. With custom backends there is more leeway when it comes to dealing with finances. [18]

Another major disadvantage of Backend as a Service is that it leaves software applications highly dependent on the vendor, also known as a vendor lock-in. This is bad because of the lack of control over the underlying infrastructure the application is built upon. This lack of control can also mean there is no way to guarantee that the BaaS platform will not suddenly shut down. This can be problematic as it can lead to entirely migrating the backend to a new provider or even worse building the entire project from scratch. An example of such an incident was when Parse, a Mobile BaaS acquired by Facebook was shut down in 2016, proving such a scenario is in fact a possibility. [18]

3.5 Progressive Web Apps (PWAs)

Progressive Web Apps are applications that utilize modern technologies, APIs, and development patterns to enhance ordinary applications by providing native-like user experience. In simple terms, PWAs do a good job integrating the best qualities of both web and native applications. For example, web apps are easily discoverable, sharable and can be accessed without the need for installation. [20]

However, native applications also have some advantages over ordinary web applications. For example, because native applications are built for a distinct platform, like Android or iOS, they are well integrated with the system, and thus are very fast and responsive. Another major advantage of native applications is that once they are installed, they have the capability of working offline. Native applications offer the users the ability to simply tap an icon to launch an application, as opposed to first looking for a browser, and then searching for the application. Ultimately, native apps have better capabilities and because they are well integrated to the system they run on, native apps can easily utilize mobile features such as camera, Global Positioning System (GPS), sensors, phone call and more. Meanwhile web apps have better reach, due to search engines, their cross-platform nature, and easy shareability. [20]

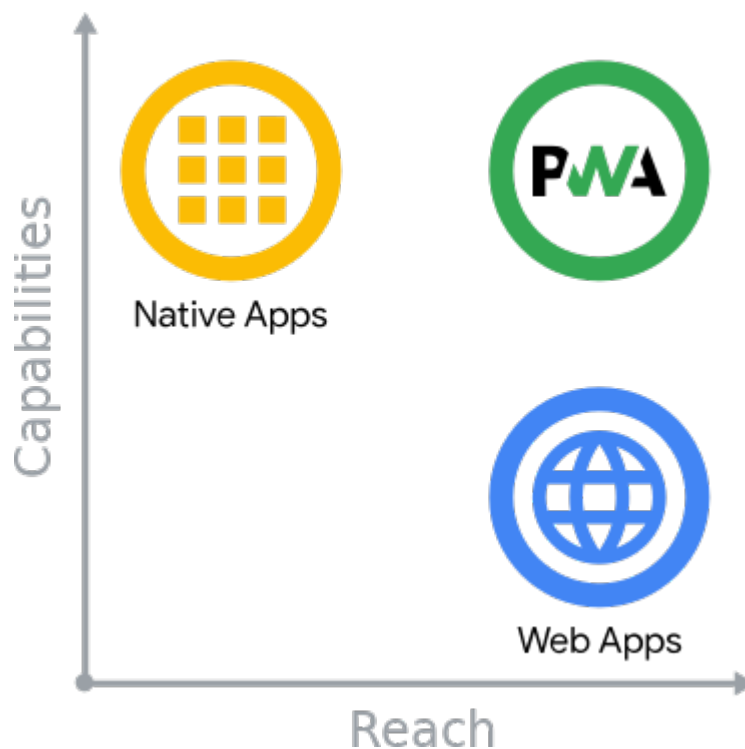


Figure 5. Capabilities vs. reach of platform-specific apps, web apps, and progressive web apps. [21]

As can be seen in figure 5, with progressive web apps, users can enjoy some of the capabilities offered by native apps while simultaneously offering the reachability of regular web apps. The idea of PWAs is not an entirely new

concept but it is a concept that has been improved using different approaches. [20]

3.6 Fundamental criteria of PWA

This section presents some of the requirements needed to build a PWA, the technologies needed, and the tools used to measure the PWA performance of applications.

For an application to be deemed a PWA, it must satisfy specific requirements or have several features implemented. To meet those requirements, and be identified as a PWA, an application needs to be, discoverable, linkable, installable, network independent, progressively enhanced, responsively designed, re-engageable, and secure.

An application with a good search engine optimization (SEO) can increase its chances of being discovered by search engines. This can help broaden the overall reach of the application and drive-up online traffic. It also makes it easier for search engines to categorize, rank, as well as provide metadata for browsers to use for other use cases. [20]

Any metadata related to an application can be specified using `<meta>` tags inside HTML `header` blocks. Similarly, a common web standard known as web app manifests can be used to include other relevant information. The web app manifest provides details about a web application in a JavaScript Object Notation (JSON) formatted file. The JSON text file can contain key information, such as `display`, `name`, `icons`, `theme_color`, and numerous more. [20]

This is a standard feature of web applications, but a powerful one. It eliminates the necessity for an app store or an installation procedure by simply providing the ability to locate an application using a unique Uniform Resource Locator (URL). [20]

Applications that are installable are easy for users to find and use. This added capability can enhance the overall experience of users. Installed applications

have the benefits of appearing on launch surfaces, on a start menu, in an applications folder, on a home screen, and in content sharing sheets. This is one key feature that is a fundamental part of the native app experience. [22]

Modern web applications can achieve the same experience in Chrome via in-browser installation. This is possible if the web app has not been already installed before and is being served over Hypertext Transfer Protocol Secure (HTTPS). The app also needs to contain a service worker registered with a fetch handler, and a web app manifest including `name`, `short_name`, `icons`, `start_url`, `display`, and `prefer_related_applications`. The `name`, and `short_name` properties define the name of the application displayed to the user. The `icons` property is an array of objects containing image files used as an icon for different contexts. For instance, to be used as a launch icon or to represent the app amongst a list of other apps. The array must include a 192px and 512px icon.

The `start_url` specifies the URL loaded when the app is launched. The `display` property determines the preferred display mode for the application, and it is recommended to be set as `fullscreen`, `standalone` or `minimal-ui`. Lastly, the `prefer_related_applications` property is a boolean value that specifies whether the browser should prompt an installation banner for a native app or PWA. Therefore, this property must be set to `false` to allow for installation.

Chrome was selected as an example as it was the preferred option for the project due to its availability. Other web browsers have similar conditions for enabling in-browser installation but with minor differences. For instance, on Opera, the `minimal-ui` mode is not supported, so instead it will fall back to `browser` mode which uses normal browser UI. In addition, Opera does not yet support `theme_color`, or `background_color`. For Samsung Internet it is suggested to set the display property to either `standalone` or `fullscreen`, and an icon where the size is at least 144x144 must be included. Lastly, if Firefox for desktop is being used then PWA installation is not supported as of 2021.

The basic idea behind network independence is to have content that can be present on the page even when the network is unreliable or unavailable. When

users revisit a site or try to access content on a page that had previously been accessed, the content is available even when the network connection is poor. [20]

Progressive web apps can achieve this by using a few technologies. Service Workers act as a proxy existing in a separate thread from the main browser thread, meaning they sit between web applications, the browser, and the network [23]. A service worker takes on the form of a JavaScript file with the ability to intercept requests and perform certain actions based on the network state.

Additionally, a Service Worker can update, retrieve resources from the Cache API, and deliver push notifications [23]. The Cache API is persistent storage for request and response object pairs and can be utilized to store site resources, such as assets, for requests made offline [24]. In combination with offline data storage technologies such as indexedDB and Web Storage, progressive web apps can deliver a seamless offline experience. [20]

Progressive enhancement is a design philosophy or methodology that encourages starting an application with a solid foundation as to provide essential content to as numerous users as possible. The goal is to provide a simple but functional experience for users on older browsers, or devices with narrow capabilities while simultaneously providing the best possible user experience for users on newer browsers or devices with increased capabilities by using feature detection to progressively enhance an application feature. [20]

Responsive web design is an approach to designing that aims to create web applications that look great on all devices, by adjusting according to different screen sizes and viewports. This can be achieved using media queries, which allow defining entirely different styles for varying browser sizes. For instance, a media query can change the `height` of a `div` element or change the font size of some text on a page depending on the breakpoints specified. Breakpoints are pixel values defined by designers or developers to specify where a change in style will occur. [25]

One important capability that used to be exclusive to native applications was their ability to re-engage users with updates and alerts to new contents using push notifications. However, a combination of new technologies has made this a possibility for modern web applications, such as Service Workers for pushing notifications to the user's device, the Push API for receiving messages from the server despite the application not being in the foreground, and lastly the Notifications API for showing messages to users even when users are not actively using the web browser.

Building a web application that is secure can have several benefits. For instance, the encryption layer is enabled, and data is protected from malicious sources. Moreover, it can help build trust with web visitors, and improve web ranking. [26]

4 Firebase

Firebase is an application development platform. It was founded in 2011 but was later acquired by Google in 2014. Today it is a major solution for web and mobile development. It offers various products for building, releasing, monitoring, and optimizing applications. Some of the services provided by Firebase include Cloud Firestore, Realtime Database, Cloud Storage, Cloud Functions, Cloud Messaging, Authentication, Hosting, Crashlytics, and A/B Testing. This paper only covers a few of these. [27]

4.1 Realtime Database and Cloud Firestore

4.1.1 Realtime Database

Firebase offers two types of databases that can synchronize data in realtime. The Firebase Realtime Database is a cloud-hosted, not only structured query language (NoSQL) database that can store data in JSON and synchronize in real time across all users. Unlike the typical HTTP request and response model, the Firebase database uses data synchronization. As a result, anytime data is changed, devices that are connected receive an update within milliseconds.

Another key capability is its ability to work offline. The Firebase SDK preserves data locally, which means applications built with Realtime Database remain responsive while offline. When a device regains connection after being in offline mode, Realtime Database can locally synchronize remote changes that occurred while the client was in offline mode. [27]

More importantly, the database can be accessed from client devices, which means server setup is not required. Database validation, and security rules can be set directly from the Firebase Command Line Interface (CLI), or Firebase Console. Unlike relational databases, the Realtime Database is a NoSQL database that scales horizontally, handles large data sets, and executes quick operations [28]. Moreover, It can support 200,000 concurrent connections, and has a 1000 writes per seconds rate limit. If the specified limits are exceeded, multiple databases can be set up. The structure of the database is JSON, and it is formatted as a key-value pair. It is supported by Android, iOS, C++, JavaScript, Unity, Flutter and more. [27]

4.1.2 Cloud Firestore

Similar to the Realtime Database, Cloud Firestore is also a cloud-based NoSQL database. It offers the same offline support, data synchronization, and it is designed to make data querying efficient. Moreover, the Cloud Firestore has a flexible data structure, and data is stored inside documents containing fields with corresponding key-value pairs, which are then organized as collections. An example of documents as seen from the Firebase console is presented in figure 6. [29]

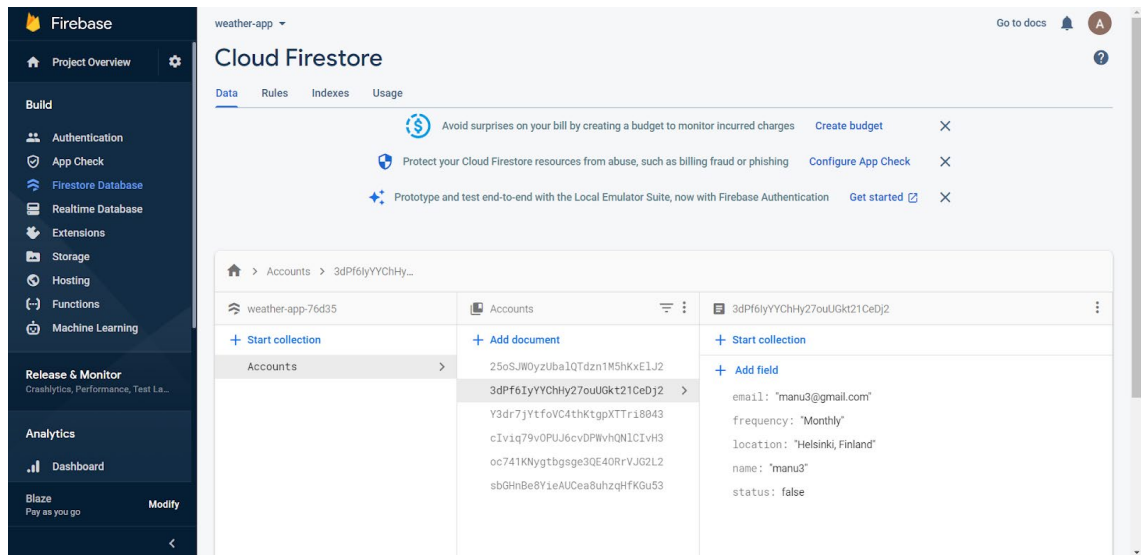


Figure 6. An example of a Cloud Firestore database from the case project.

Documents support various data types ranging from simple strings and numeric values to complex data types, such as nested objects, also known as maps. Additionally, documents can contain subcollections, which can form a hierarchical data structure. Documents are very similar to JSON, but they support more data types, and have a size limit of 1MB. The Cloud Firestore is different from relational databases in that it has no schema, which means it offers more liberty over fields or data types that are stored inside documents. Lastly, Cloud Firestore supports simple queries, complex queries, filtering, limitation, and sorting data. [29]

4.2 The Difference between Cloud Firestore and Realtime Database

Choosing between Cloud Firestore or Realtime Database can be difficult as they both appear to have the same capabilities. However, the two databases are not the same. Some of the differences between the two are highlighted in table 1.

Table 1. The Differences between Cloud Firestore and Realtime Database. [30]

	Realtime Database	Cloud Firestore
--	-------------------	-----------------

Data model	Can be challenging to manage complex hierarchical data at scale	Easier to manage complex hierarchical data at scale. Can be achieved using subcollections inside documents
Realtime and offline Support	Offline support only for iOS and Android clients.	Offline support for iOS, Android, and Web clients.
Client Presence	Client connection status is supported	Client connection status is not supported natively
Querying	• Supports filtering and sorting but not simultaneously. • Supports deep queries which means the response will include the whole data subtree. • Indexes are not a necessity; but there can be performance loss for specific queries as the data size increases	• Supports filtering and sorting a property at once by chaining filters • Queries are not deep hence only the level at which is queried is retrieved and not subcollections. • Supports default indexing, which means the performance of queries depends on the result size, and not the size of the data requested from.
Reliability and performance	• Database can only be configured regionally. • availability within a region.	• Database can be replicated across different zones and regions.

	• Ideal when the user base is in a single region.	• Ideal when the user base is from a different region.
Scalability	• Scales to roughly 200,000 simultaneous connections per database. • For limits beyond it is required to distribute data across multiple database instances • Supports 1000 writes/second	• Scales automatically • It has a scaling limit of about 1 million simultaneous connections and 10,000 writes per second.
Security	• Secured with Realtime Database rules • Read and write access rules applied at parent node will be applied to child nodes. In other words, it cascades. • Validation and authorization are distinct. Data is validated by making use of a validate rule	• Web and mobile SDKs are protected with Cloud Firestore Security rules • Rules do not cascade without a wildcard - a rule set for matching deeply nested subcollections is used • Queries can fail if the result contains data that a user does not have access to

As demonstrated in table 1, selecting the right database for a specific project can depend on multiple factors. Some key points to consider before selecting

between the two databases include the role of the database, data operations, the data model, availability, offline queries, and database instances.

As far as the role of the database, the usage of the database should be known. If the database is used for simple queries and data synchronization, then Realtime Database is recommended. Otherwise for complex queries, then Cloud Firestore is preferred. It is also good to know the operations that will be made on data. For extremely large data sets that require more reads than write operations, Cloud Firestore is favored, but for large datasets with frequent updates, Realtime Database is suggested.

The structure of the data can also be used to determine an appropriate pick between the two databases. If the structure is a JSON tree, then Realtime Database is more suitable whereas if the preferred structure is as collections containing documents, then Cloud Firestore would be more favorable. In some cases, the uptime of the database might even be crucial. Therefore, if a critically high uptime is of importance, then Cloud Firestore is suggested. Otherwise for high but non-critical uptime, Realtime Database is recommended. Moreover, if poor connectivity is of main concern, then it might be good to know the frequency at which queries are made on devices with poor connectivity. If frequent, Cloud Firestore is advised, and if not, then either one would be preferable. Lastly, if the usage requires beyond the capabilities offered by a single database, then the amount of database instances that can be created might be important. For multiple instances, Realtime Database is preferable. Otherwise, either one would suffice.

4.3 Authentication, Cloud and Hosting

4.3.1 Authentication

Most applications today require a way to confirm a user's identity. Verifying a user's identity enables an application to safeguard user information, personalize user experience and provide a way to permit access to confidential resources by checking the authentication state of users. [31]

Firebase Authentication provides a couple of methods for authenticating users to an application. The FirebaseUI auth is an open-source library that offers an easy solution for handling authentication. It uses pre-made but customizable UI to handle authentication flows for different sign-in methods. FirebaseUI is the recommended way to integrate a sign-in system into an app because it helps eliminate unnecessary boilerplate code, but it is also possible to authenticate using an SDK. [31]

The firebase authentication SDK also offers a way to create and manage users, but it must be set up manually. Similar to FirebaseUI it supports authentication with email address and password, phone number, and third-party identity providers, such as Google, Facebook, and many more. Additionally, it provides a way to integrate custom auth systems, as well to anonymously sign-in users by creating temporary accounts that can be upgraded to regular accounts if anonymous decide to sign up later. [31]

The Firebase authentication system also handles password recovery, account verification, and account linking. Account linking allows users to sign-in using multiple auth providers. This is made possible by connecting auth provider credentials to existing accounts. It is also entirely possible to customize message templates used for email verification, and account recovery. [31]

4.3.2 Cloud Storage

Cloud storage is a solution built for storing and serving user-generated content such as photos, videos, audios, and documents. It is backed by Google Cloud storage, which means user-generated content can be accessed through Firebase or Cloud Storage. With the help of the Firebase SDKs, developers can upload or download directly from clients. Additionally, it is also possible to access the same content on the server with the help of Google Cloud Storage APIs. If necessary, this gives developers the ability to perform server-side operations such as image filtering or resizing. [32]

Moreover, the Firebase SDKs for Cloud Storage has a few key capabilities. For instance, Cloud Storage can perform operations regardless of network quality. If

the connection is poor during uploads or downloads, the operation can be resumed from where it left off. The Firebase SDKs for Cloud Storage is also tightly coupled with Firebase Authentication for authenticating, and authorizing users. In combination with Cloud Storage Security Rules, Firebase offers a secure storage that allows developers to write rules that limit upload size, validate data, or restrict requests based on metadata. Lastly, Cloud Storage is built to be scalable, meaning migration to other providers is not a requirement. [32]

4.3.3 Cloud Functions

Cloud Functions is a serverless framework that can be used to run backend logic in response to events triggered by Firebase services, and HTTP requests. Cloud Functions can be integrated with various Firebase features by using the admin SDK. It requires no maintenance and is fully protected from the client which makes it secure and private. [33]

Developers can deploy code with Cloud Functions directly using the command line. Once a function has been deployed, Google's servers will instantaneously start to manage the function. The function will fire in response to HTTP requests or in the case of a background function, it will listen to events, and run the function once it has been triggered. Each function runs in a separate environment with its own configurations. If the load of an application increases or decreases, it will quickly scale the amount of virtual server instances required to run the function. If the function is idle, updated or deleted, instances will be cleared up and replaced by new instances. Additionally, when there is a deletion, any connections with the event provider are removed. [33]

4.3.4 Hosting

Firebase Hosting is a service used to deploy web applications, and can support static or dynamic content, as well as microservices. It is built with free Secure Sockets Layer (SSL), so the connection over which the content is served is

secure. Before deployment or updates, Firebase provides a Local Emulator Suite for testing from a locally hosted URL. Deploying an application with Firebase Hosting is as easy as writing a single command. Once deployed, content is served from the closest edge server on a global delivery network (GDN), which means content will be delivered fast no matter the location of users. [34]

Firebase Hosting also has configuration options which make it possible to customize hosting behavior. Configurations can be specified inside a `Firebase.json` file which is created when initializing Firebase locally. With hosting configurations, it is possible to specify files to deploy, customize 404/not-found pages, or set up redirects for deleted or moved pages. [34]

5 Supabase

Supabase is a platform that offers a collection of tools for building applications. It provides backend services, such as authentication, database, file storage, auto-generated APIs and edge functions. The Supabase client library is supported in JavaScript, Python, and Dart. The platform was founded in 2020 and is marketed as an open-source alternative to Firebase. [35]

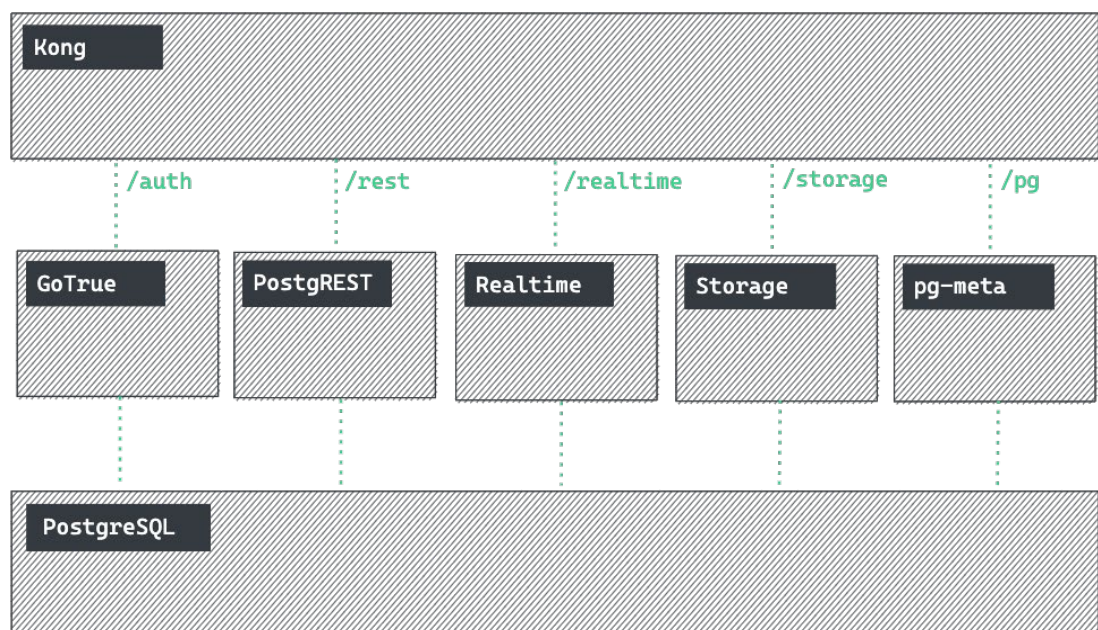


Figure 7. The Supabase architecture.

As illustrated in figure 7, Supabase is created using a combination of different open-source tools that are scalable. If the features already provided by Supabase are inadequate, supabase offers the option of self-hosting.

5.1 Database and Authentication

Every project created in Supabase includes a full Postgres database. Since the database is built on Postgres, it is relational. This means that data can be organized into tables that are linked based on defined relationships. The items inside the tables can be organized as rows and columns. Supabase offers a few ways of creating tables. The easiest way to create a table is directly from the dashboard using the intuitive table view provided by Supabase but it is also possible to do so by running SQL queries. When creating a table each column must have a predefined data type. Supabase offers default types but it is also possible to create custom types if the ones already provided are insufficient.

Database tables are secured using Row Level Security (RLS) policies. Since data can be accessed directly from the client using the anon key provided by Supabase, it is important to restrict access to private tables using RLS policies. This can be achieved by running code inside the SQL editor or via the Supabase dashboard. The policies can be as complex as required.

Supabase also supports SQL functions that exist inside the database. A database function can be created using SQL or from the dashboard. After a function is created it can be executed inside the database or directly from the client library. A database function is best suited for handling data intensive operations.

The platform also includes other additional features, such as a realtime engine that is extended by postgres for listening to database changes. It also offers extensions that can be enabled to further expand the functionality of the database. Moreover, Supabase self manages data backups, and allows data imports from comma-separated values (CSV) or excel spreadsheet. It also auto-

generates self-documenting, secure, fast, and scalable APIs directly from the database schema. [35]

The three APIs generated by Supabase include RESTful, Realtime, and GraphQL (Graph Query Language). The RESTful API handles CRUD operations and can be useful for fetching data from multiple tables by using deeply nested joins. The Realtime API listens to database changes and the GraphQL API functions as a GraphQL endpoint which is currently in beta.

Supabase also has a system for authorization which offers several ways of authenticating users. For instance, users can be authorized with email and password, phone number, magic links (one-click sign-in), or third-party social providers. For the third-party authentication, Supabase offers a great deal of providers to choose from. Third party authentication can be enabled from the settings tab of the authentication page by providing a `client ID` and a `secret` for the selected provider. When a user signs up, Supabase automatically creates a user's table with unique ids which makes user management, and authorization easier. For authorization, RLS and Policies can be used to set complex SQL rules. [35]

5.2 Supabase Storage and Edge Functions

Similar to Firebase, Supabase provides space for storing and serving media files such as images, and videos. The Supabase storage uses buckets which serve as containers for files and folders. A bucket can be useful for separating files based on access rules. For instance, files that are accessible by everyone can be stored inside a public bucket and private files can be stored inside a private bucket. Buckets can be created using the Supabase Dashboard, an SQL editor or the Supabase client library. Figure 8 showcases a bucket created for storing avatars using the Supabase Dashboard.

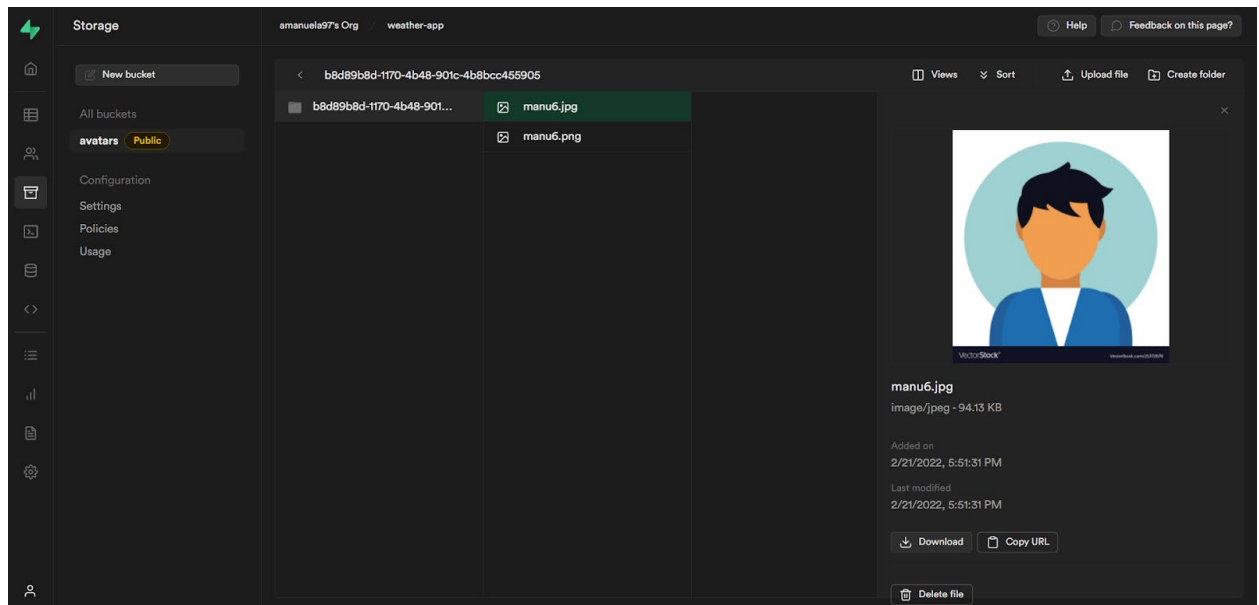


Figure 8. An example of a Supabase Storage from case project.

The images stored inside the avatar bucket, as shown in figure 8, are organized under folders with user ids as names for easier access. To secure buckets, operations made to files can be restricted by adding policies with different security rules. Moreover, files can be uploaded, downloaded, sorted or deleted directly from the dashboard. [35]

In March 2022, Supabase launched edge functions. Edge Functions are serverless Typescript functions that are located geographically closer to end users. Edge Functions can be integrated with other Supabase services or third-party libraries to perform tasks such as credit card payment processing. Edge Functions are best suited for use cases that require low-latency.

At the time of writing, in 2022, this service is new and is still in the experimental stage, hence it is not recommended for production use. The Supabase CLI can be used to create and deploy Edge Functions or manage environment variables. Edge functions can be deployed locally or to production. When deploying to production, the `deploy` command automatically bundles the edge functions. Once deployed, the Functions section of the dashboard offers a couple of debugging tools. The first debugging tool is called invocations and is used to display requests and responses for each execution. The second

debugging tool logs all platform related events such as errors and deployments.[35]

6 Implementation

For the case project, two identical applications were built using two different backend service providers. This chapter details some of the steps taken to create the case project. Furthermore, it aims to highlight the main distinctions in implementation of an application using Supabase and Firebase. It describes the functionalities of the project application and briefly mentions other technologies, and software tools required to facilitate the creation of the product.

6.1 The case project

6.1.1 Case project description

The goal for the case project was to create an application that would send an email to users, including weather information for the following day. Users would have the options to specify the location for the weather information, and how frequently they would want to receive the message. In addition, users also can disable the option to receive any weather updates. To make the application tailored to individual needs, users would be required to provide personal information by signing up.

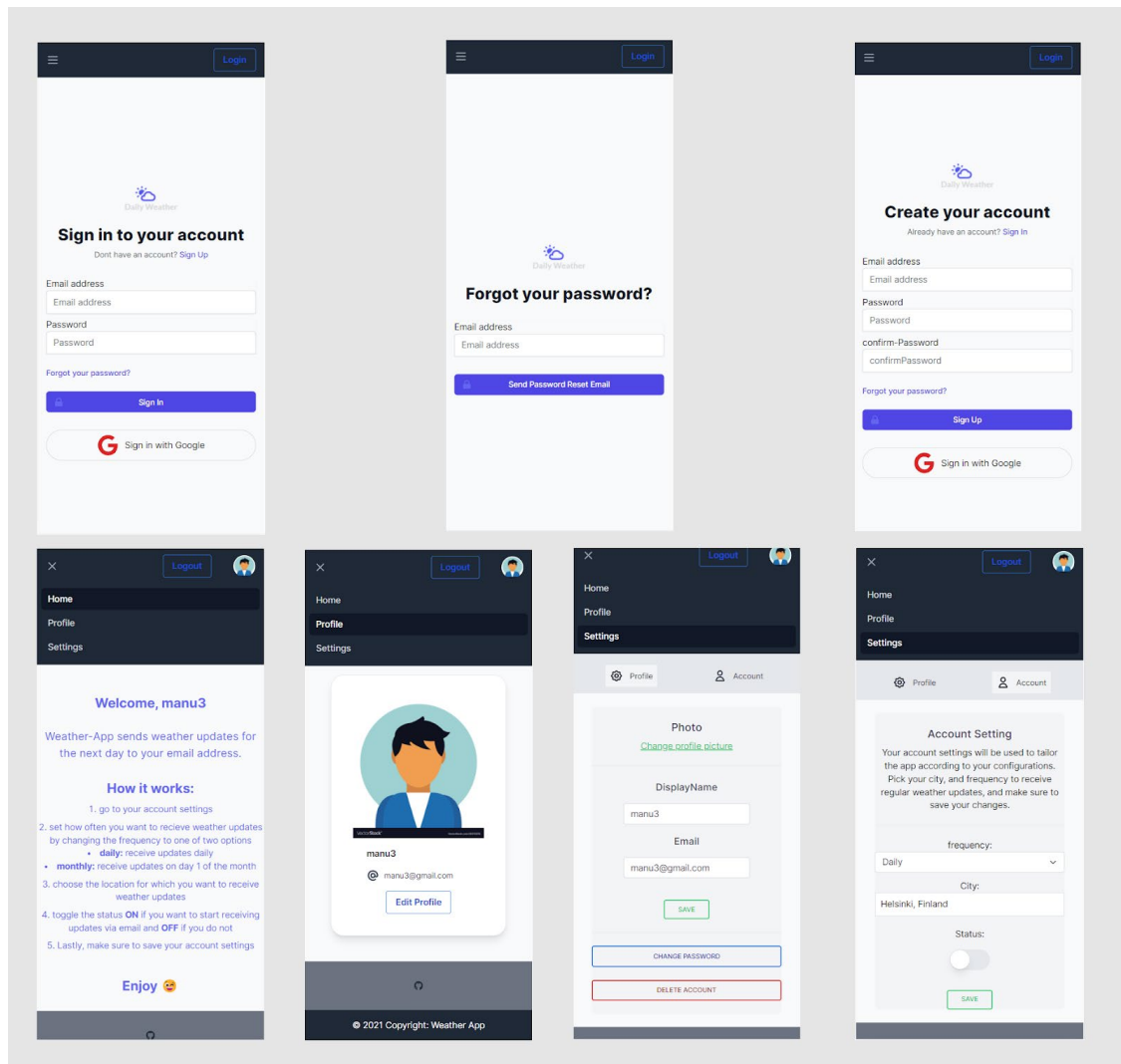


Figure 9. A screenshot of the different parts of the application on mobile.

Visually, the application has a sign-in form as a start-up screen as seen in the top left of figure 9. The page has links pointing to forms for registering and recovering passwords. If a user logs in successfully, the user is redirected to the home page depicted in the bottom left of figure 9. For navigating to different parts of the page, a navigation bar is included with different tabs.

The profile tab, second from the bottom left of figure 9, is used to navigate users to the profile page where the user can see a profile image, display name, email, and a button to modify the profile details. The settings tab is for navigating users to a page with a side bar including two more tabs which lead to an account settings form, and a profile settings form. With the profile settings form, located third from the bottom left of figure 9, users can change their profile image,

display name, and email. Users can also use the form to delete their account or change their password. Lastly, from the account settings form, located first from the bottom right of figure 9, users can configure the behavior of the weather app by setting how frequently they want to receive an update, what city they want to receive an update for, and whether they should even receive anything at all.

6.1.2 Project environment and file structure

The development environment for the project uses a few programming tools to set an efficient workspace. The selected code editor for the project was VS code¹ which is a popular work environment for building and debugging applications. The project also requires Node.js² to be installed. Node.js is a runtime environment that allows JavaScript to be run on the server. Node.js has a package manager known as node package manager (NPM)³, which is used for installing libraries for the project. For styling, tailwind is installed then tailwind directives are added in a CSS file. A `tailwind.config.js` file is created for configurations and specifying files where tailwind⁴ is required. ESLint⁵ is also added for code formatting and adherence to a specific style guideline.

¹ <https://code.visualstudio.com/>

² <https://nodejs.org/en/>

³ <https://www.npmjs.com/>

⁴ <https://tailwindcss.com/>

⁵ <https://eslint.org/>

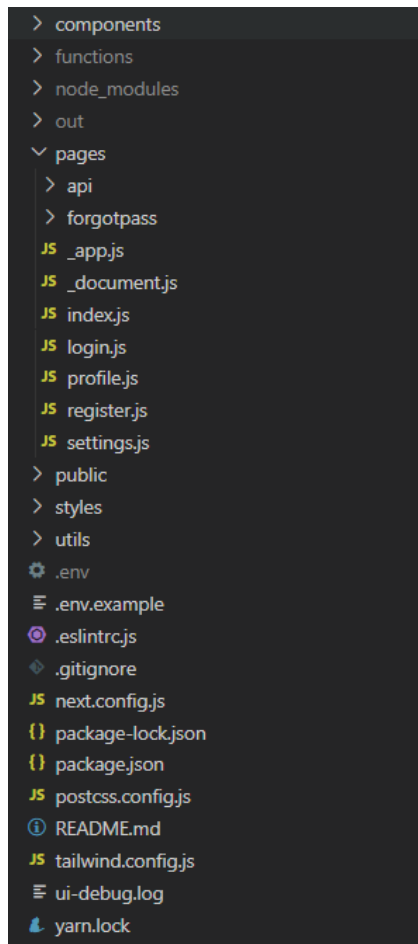


Figure 10. The file structure of the project

Figure 10 illustrates the general code structure which follows the automatic setup provided by a NextJS project. The `components` folder is where the different components used in the application are defined. The `pages` folder is where the routes for the pages are defined based on the filenames. The folder also includes an `api` folder where any API endpoints for the application are specified. All static files are stored inside the `public` folder for NextJS to serve. The `styles` folder contains a CSS file where tailwind directives are defined. Lastly, the `utils` folder is where all JavaScript logic is defined.

6.1.3 Implementing the UI

The application utilizes forms, dropdowns, modals, a navbar, buttons, and many more UI elements. To facilitate this, customized TailwindUI components were used. TailwindUI is a component library that provides professionally designed

responsive UI components. At its core, TailwindUI uses tailwind, which is a CSS framework with utility classes for rapidly building UI elements. A few advantages of tailwind is that it is written using inline styling, which makes CSS classes and ids redundant, and speeds up the UI building process since it has built-in classes that are ready for use [36].

The components in TailwindUI are designed for tailwind CSS version 2 and 3. Therefore the project uses tailwind version 3. TailwindUI relies on Headless UI for its UI components, and to power the interactive behavior it offers. It also relies on Heroicons for icons, so both libraries need to be installed before using it. Once TailwindUI was fully integrated, the application was built with responsive components to adjust to different screen sizes. The distinction between the apps built with Supabase and Firebase is mainly in the business logic, and therefore both applications have an identical appearance.

6.2 Firebase implementation

6.2.1 Getting started with Firebase

Before using Firebase in the project, a project must be created through Firebase Console. Firebase Console can be accessed by signing in with a Google account or creating a new one. Once successfully logged in, the dashboard shown in figure 11 is presented.

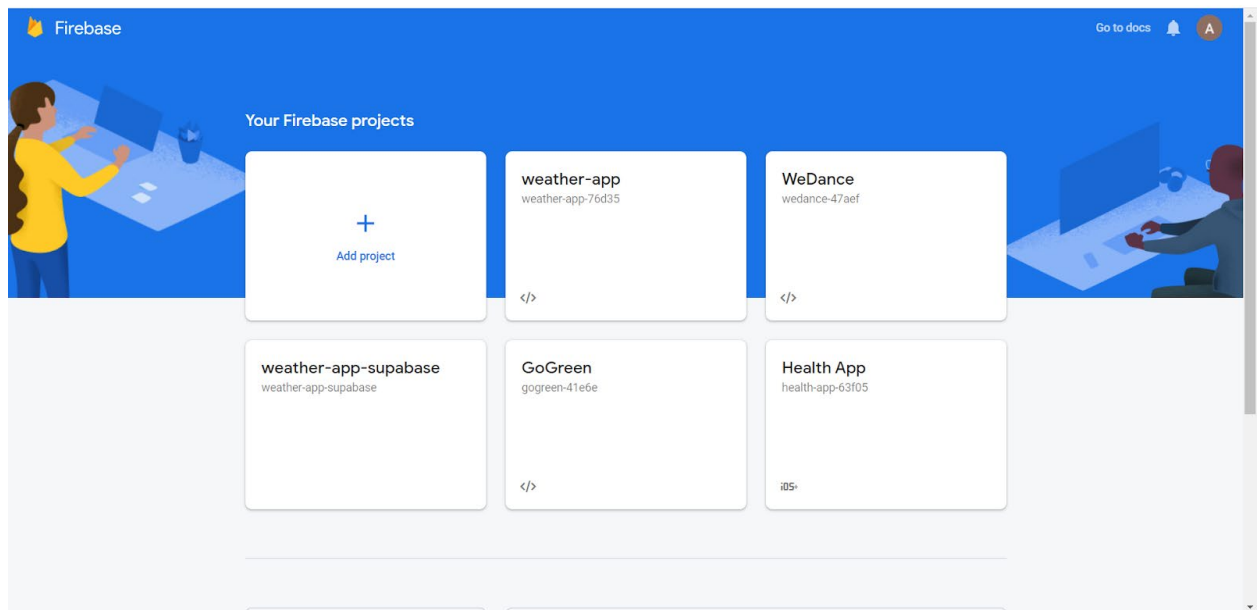


Figure 11. The Firebase dashboard for creating a new project

Clicking on the add project button shown in figure 11 will open a pop-up window that asks for the project name, whether Google analytics should be enabled and what the country of organization for the project should be. Google analytics was disabled for this specific project since it was not necessary. After providing the required details, the Project dashboard will be visible. To integrate Firebase into the app, the cog wheel icon was first clicked and then the web app icon was selected from the project settings page. Firebase will then generate the code snippet shown in listing 2.

```
const firebaseConfig = {
  apiKey: "123",
  authDomain: "app.firebaseio.com",
  projectId: "app-123",
  storageBucket: "app-123.appspot.com",
  messagingSenderId: "123",
  appId: "1:123:web:123b"
};
const app = initializeApp(firebaseConfig);
```

Listing 2. A code snippet containing a Firebase configuration

Before using the `firebaseConfig`, the `npm install firebase` command is run in the terminal inside the root project folder. Then the Firebase configuration was used inside the project to initialize a new Firebase application as illustrated in listing 2. For the case project, the Firebase SDK was initialized as a client with Firestore, Authentication, Storage, and a Google authentication provider.

```
import { initializeApp, getApps, getApp } from 'firebase/app'
import { getAuth, GoogleAuthProvider } from 'firebase/auth'
import { getFirestore } from 'firebase/firestore'
import { getStorage } from 'firebase/storage'

let app
if (!getApps().length) {
  app = initializeApp(process.env.NEXT_PUBLIC_FIREBASE_CONFIG)
}else {
  app = getApp('weather-app')
}

const db = getFirestore()
const auth = getAuth()
const storage = getStorage(app)
const provider = new GoogleAuthProvider()
provider.setCustomParameters({ prompt: 'select_account' })
//View your data across Google Cloud Platform services
//provider.addScope('https://www.googleapis.com/auth/contacts.readonly')

export { db , auth, provider, storage }
```

Listing 3. Firebase client initialization.

Listing 3 shows the Firebase client being initialized using the configuration details stored as an environment variable under the key `NEXT_PUBLIC_FIREBASE_CONFIG`. The app is only being initialized if an instance does not already exist. Otherwise, the `getApp(name)` function is used to retrieve an existing `FirebaseApp`. The `setCustomParameters` function of `GoogleAuthProvider` is also set to `select_account`. This forces account selection when signing with Google.

6.2.2 Implementing authentication and user management with Firebase

Before using Firebase authentication methods, the required sign-in providers must be enabled. For this project, email/password, and Google sign-in were enabled. Firebase provides methods for sign in, sign up, and password resets. The sign-in flow for Google has two modes. The first mode is a Popup mode which opens a popup window when asking users to sign into their Google

account. The second mode is Redirect mode which forces a redirect to a new page to sign-in. For this project, the Popup mode was used as demonstrated in listing 4.

```
const SignInWithGoogle = async () => {
  signInWithPopup(auth, provider).then((result) => {

    const credential = GoogleAuthProvider.credentialFromResult(result)
    const token = credential.accessToken
    const user = result.user

  }).catch((error) => {
    // Handle Errors here.
    const errorMessage = handleError(error)

    const email = error.email

    const credential = GoogleAuthProvider.credentialFromError(error)
    console.log({ error, email, errorMessage, credential })
  })
}
```

Listing 4. Google sign-in logic using Popup mode.

In addition, Firebase provides methods, properties, and observers for user management. The `onAuthStateChanged` observer listens for the authentication state of the user and returns the currently signed-in user. This allows for authorizing a user based on sign-in state. Firebase provides methods for user management. For instance, methods were used to set email and password for a user, to re-authenticate a user, and to delete a user. The `currentUser` property was also useful for retrieving the profile information of a user. In the project it was used to display profile details.

To allow users to save profile photos, Firebase Storage provides a method for uploading files. Before uploading a file, a reference to its full path and name needs to be created. Once a reference is created, Firebase can upload from a file, a binary large object (blob), byte array, or string. For the case project, the `state_changed` observer was used to monitor the upload progress and display a progress bar. Lastly, to secure the images, the read and write rules depicted in listing 5 were defined using the Firebase Security Rules.

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
```

```

match /{userId}/profilePicture/{fileName} {
  allow read: if request.auth != null;
  allow write: if request.auth.uid == userId
    && request.resource.contentType.matches('image/.+');
}
}
}

```

Listing 5. Security rule for Firebase Storage.

The Firebase Security Rules operate by first matching a pattern to a database path, then using a custom condition to grant access rights to data. The security rule defined in listing 5 allows users to read from the path only if they are authenticated. To write or edit a file, the user must be an owner of the file, and the only content type that is supported is of image type.

6.2.3 Implementing the main functionality with Firebase

The main functionality of the application is to be able to send weather details for a specific city. To achieve this, data about the specified city and information on how frequently the updates should be sent was collected and stored. Then a scheduled function was created to retrieve the stored information and send weather updates to users.

A form was created to gather the required details. To implement a selector for picking cities an NPM package called react-google-autocomplete was used. The package easily integrates google places autocomplete service with React. After implementing the form and once the frequency, city and status fields were filled, submitting the form would trigger a function that would validate the selected city as shown in listing 6. The validity of the city was checked to see if the weather API had information for the selected city. The weather API used for the project was Visual Crossing⁶.

```

const isLocationValid = async () => {
  const url =
`https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/ti
meline/${location}/tomorrow?unitGroup=metric&key=${process.env.NEXT_PUBLIC_WEA
THER_API_KEY}&contentType=json`
  const options = {

```

⁶ <https://www.visualcrossing.com/>


```

    method: 'GET',
    headers: {
      'Content-Type': 'application/json'
    },
  }
}
try {
  const response = await fetch (url, options)
  const data = await response.json()
  if(data?.address){
    return true
  }
  return false
} catch (error) {
  console.clear()
  return false
}
}

```

Listing 6. Function used to validate a city using Visual Crossing API.

The next step was to create two scheduled functions where one would run daily, and the other monthly as depicted in listing 7. Firebase scheduled functions support Unix Crontab and App Engine syntax for specifying the time. The functions were defined, and Firestore was initialized using the admin SDK. Inside the scheduled functions, account information was retrieved based on the status set for the weather update. Once all the information was retrieved from the database, it was iterated, and the email address was used to send the weather to corresponding users. For sending email messages, Nodemailer⁷ was used. Nodemailer is a Node.js module which allows sending email messages using email credentials.

```

export const dailyEmail = functions.pubsub.schedule('0 0 * * *')
  .onRun(handleDailyTask)
export const monthlyEmail = functions.pubsub.schedule('0 0 1 * *')
  .onRun(handleMonthlyTask)

```

Listing 7. Scheduled functions used to run tasks.

Finally, once all the functionalities were complete the app was deployed using Firebase Hosting. To achieve this, the Firebase CLI was used first to set Firebase configurations. Lastly, the deploy command was used to deploy the project using Firebase CLI. When the deployment is complete, the CLI logs the application URL. The default behavior is that Firebase generates a domain

⁷ <https://nodemailer.com/about/>

name based on the project name, but it is also possible to add a custom domain name if needed.

6.3 Supabase implementation

6.3.1 Getting started with Supabase

The first step to getting started was signing in to Supabase with a GitHub account. Once signed-in clicking the New Project green button opens a box asking for an organization, name, database password, and region. The default organization, and the closest region were selected. The rest of the box was filled with the project name, and a strong database password. Finally, clicking the Create new project button generates a new project.

Before initializing the client, a profile table was created from the Table Editor section of the sidebar. Once a database table was created, Supabase automatically creates an API for the client with auto generated documentation. The client can be set up by directly copying the code from the generated documentation.

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
const supabase = createClient(supabaseUrl, supabaseKey);

export {supabase};
```

Listing 8. Supabase client initialization.

A URL `service_role` and `anon_key` is also provided from the API settings sidebar. The `service_role` key is what gives full access to the data and can bypass any security policies. Therefore, the `service_role` key should not be used client side. In listing 8 the Supabase `url` and `anon_key` are passed as arguments to the `createClient(url, anon_key)` function to initialize Supabase.

6.3.2 Implementing authentication and user management with Supabase

To be able to authenticate users, Google, and email sign up need to be enabled from the Supabase settings sidebar. To successfully enable Google sign up it is also required to provide a Google client id and secret. To achieve this a Google project needs to be created on the Google Cloud Platform Console. Afterwards, the client id and secret can be found from the credentials page of the Google APIs console.

```
const {user, session, error} = await supabase.auth.signUp({
  email: 'example@email.com',
  password: 'example-password',
})
```

Listing 9. User creation syntax using Supabase.

The implementation of authentication with Supabase has a lot of similarity with Firebase. The main distinctions worth noting were the difference in syntax and the deletion of users. The first distinction is that the syntax of a Supabase function includes a response that is destructured into an error, and data property as demonstrated in listing 9.

```
try {
  const result = await createUserWithEmailAndPassword(email, password);
  user = result.user
} catch (error) {
  console.log(error);
}
```

Listing 10. User creation syntax using Firebase.

Listing 9 and 10 demonstrate the syntactical difference in code between Firebase and Supabase. Another distinction is how user deletion is implemented in Supabase. To implement user deletion, the `deleteUser(userId)` function must be called only on the server. This contrasts with Firebase where the deletion can be executed client side. To achieve this, an API endpoint was ultimately created where the function can be executed.

6.3.3 Implementing the main functionality with Supabase

At the time of implementing the main functionality in May 2022, Supabase did not have functions as a feature. Supabase Edge Functions are now supported but the limitations faced were that it was still in the experimental stage and only supported POST responses. Therefore, to achieve sending weather updates to users periodically, a Postgres extension named `pg_cron` was installed. The extension would run periodic jobs within a Postgres database which was exactly what was needed for this use case. Additionally, an HTTP extension was also enabled to allow making HTTP requests inside the database a possibility.

```
create or replace function send_weather()
returns void as $$
  const profiles = sql('SELECT * FROM profiles WHERE frequency = $1 and status
= $2', ['Daily', true]);
  for (var i = 0; i < profiles.length; i++) {
    const loc = profiles[i].location;
    const email = profiles[i].email;
    const displayName = profiles[i].user_name;
    const weather = exec('get_weather', [loc]);
    if(!weather) return
    const body = `
Hey ${displayName || ''}! Your weather update for tomorrow \n
address: ${weather.address || '-'} \n
timezone: ${weather.timezone || '-'} \n
datetime: ${weather.days[0].datetime || '-'} \n
temperature: ${weather.days[0].datetime || '-'}°C \n
humidity:  ${weather.days[0].humidity || '-'}% \n
precipitation:  ${weather.days[0].precipprob || '-'}% \n
wind: ${weather.days[0].windspeed || '-'}m/s \n
conditions: ${weather.days[0].conditions || '-'} \n
`;
    const msg = {"sender": "test@gmail.com","recipient":
`${email}`, "subject": "Weather Updates", "text_body": `${body}`};
    const eml = sql('SELECT send_email_message($1)', [msg]);
  }
$$ language plv8 security definer;
```

Listing 11. JavaScript code with plv8 extension for constructing weather data, and message.

To be able to send a message from Supabase, Supabase-mailer was used in combination with a transactional email provider. The selected email provider for the case project was Sendgrid⁸. As part of the process, a Sendgrid account was created to retrieve an API key for sending messages. Afterwards, a JavaScript extension for Postgres named plv8 was added to allow writing in JavaScript.

⁸ <https://sendgrid.com/>

Lastly, all the logic for constructing the message was defined inside the Supabase SQL editor as illustrated in listing 11.

```
select
  cron.schedule(
    'weather-every-month', -- name of the cron job
    '0 0 1 * *', -- every month
    $$
      select send_weather();
    $$
  );
```

Listing 12. The cronjob scheduler for sending weather updates

The `pg_cron` extension allows scheduling Postgres commands directly from the database. The cron based scheduler uses the same syntax as a regular cron, where the `*` sign means run every time period, and where a specific number means but only at this time. The cron job illustrated in listing 12 accepts two arguments, the name of the cron job, and how frequently it is executed. In this case, it will run at midnight on every first day of the month.

6.4 Creating a PWA with NextJS

To convert the app into a PWA, a plugin for Next.js called `next-pwa`⁹ was installed. After installing the required dependency, a `manifest.json` file was created including a maskable icon, which makes the PWA icon look good on all android devices. The file was then stored inside the public directory along with all the icons referenced inside it. The manifest file contained a `theme_color`, `background_color`, `scope`, `start_url`, `short_name`, `description`, `icons`, and a `display` set to `standalone`.

```
<Head>
  <link rel="manifest" href="/manifest.json" />
  <link rel="apple-touch-icon" href="/appLogo.png" />
  <meta name="theme-color" content="#fff" />
</Head>
```

Listing 13. The code snippet required inside the `_document.js` file

⁹ <https://www.npmjs.com/package/next-pwa>

As illustrated in listing 13 a `_document.js` file was created inside the `pages` folder to reference the location of the documents manifest as well as set the icon and theme color. The remaining steps were to configure next-pwa inside the `next.config.js` file as displayed in listing 14, and to include the auto generated files by next-pwa inside the `.gitignore` file.

```
const withPWA = require('next-pwa');

module.exports = withPWA({
  reactStrictMode: true,
  env: {
    NEXT_PUBLIC_FIREBASE_CONFIG: JSON.parse(
      process.env.NEXT_PUBLIC_FIREBASE_CONFIG
    ),
    NEXT_PUBLIC_GOOGLE_MAPS_API_KEY:
      process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY,
    NEXT_PUBLIC_WEATHER_API_KEY: process.env.NEXT_PUBLIC_WEATHER_API_KEY,
  },
  images: {
    domains: [
      'tailwindui.com',
      'images.unsplash.com',
      'lh3.googleusercontent.com',
      'firebasestorage.googleapis.com',
    ],
  },
  pwa: {
    dest: 'public',
    register: true,
    skipWaiting: true,
  },
});
```

Listing 14. Configuring PWA inside the `next.config.js` file.

As seen in listing 14, the `pwa` object inside the `next.config.js` is used to customize the behaviour of the plugin. The `dest` property determines where the files generated by the plugin will be stored. In this case it is stored inside the public directory. The `register` property which is set to true tells the plugin to register a service worker. Lastly, the `skipWaiting` property ensures that a new service worker is immediately activated.

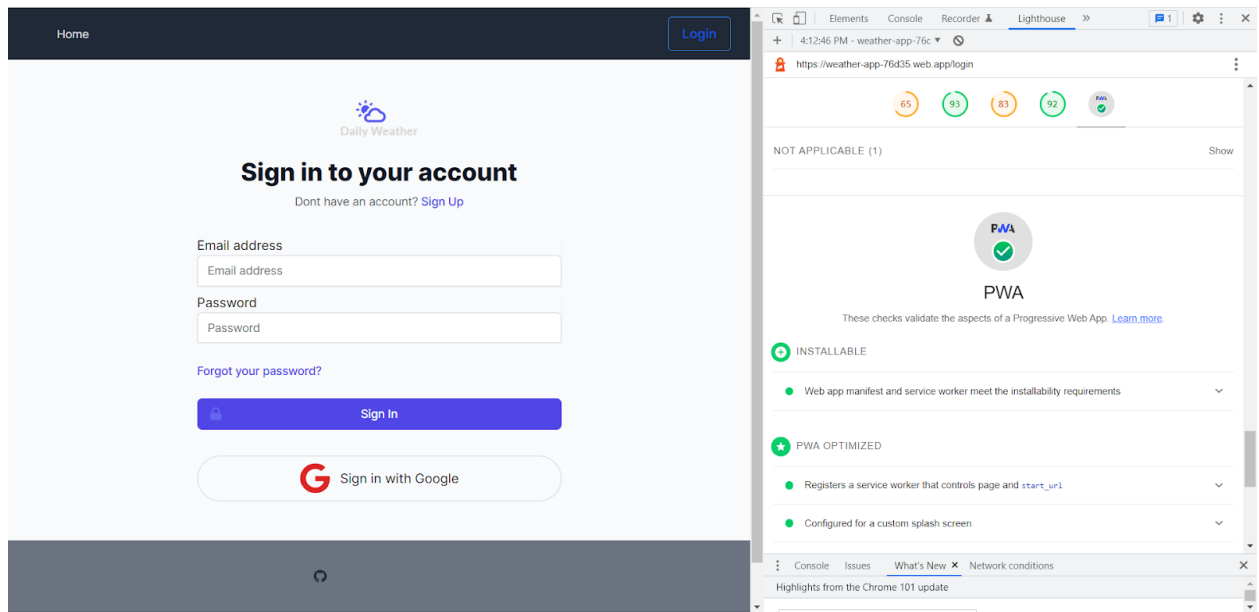


Figure 23. A Lighthouse report in Chrome DevTools.

To test whether the application was PWA, an audit was performed from the Lighthouse panel of Chrome's DevTool. Lighthouse is a free software that can perform inspections for accessibility, PWA, best practices, performance and more [37]. Figure 23 shows that the application passes all the requirements for a PWA.

7 Results

7.1 Advantages of Firebase over Supabase based on different aspects

7.1.1 Firebase features

When comparing features, the results showed that Firebase is a lot more feature rich. Firebase offers products for building, releasing, and monitoring applications as well as engaging users to applications. For instance, Firebase offers a performance monitoring tool which can help a developer gain some important insight about the performance of an application. The performance tool is capable of measuring the startup time of an application, and how long it takes a block of code to execute.

Supabase currently only offers products for building software, but even then the Supabase product catalog is not comparable to that of Firebase. In total, Firebase offers 24 products while Supabase is mainly focused on 4 key products. This is partly because, as of 2022, Supabase is still relatively new as it was first launched in 2020.

7.1.2 Firebase stability

Firebase is 8 years older compared to Supabase since it was launched to the public in 2012, for this reason it has a larger presence in the market and is known to be stable. Firebase offers a documentation that it is a lot more information dense than the Supabase documentation. Moreover, Firebase has an SDK that supports C++, JavaScript, Java, Objective-C, and Swift, whereas Supabase has a client library that supports JavaScript, Python, and dart.

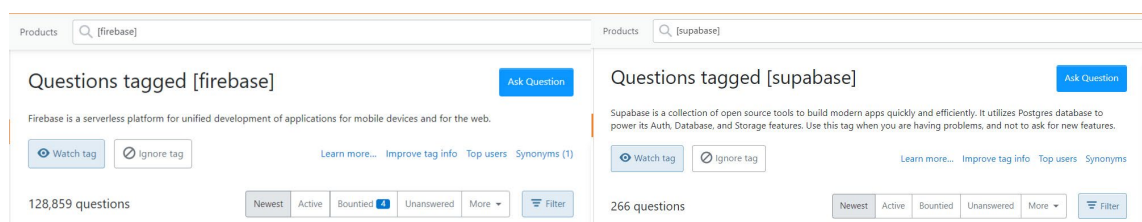


Figure 24. Firebase and Supabase search results comparison on stackoverflow.

With its longevity, Firebase has acquired a large community of users, and a strong support system. Figure 24 shows a simple search result with the Firebase tag on stackoverflow generates hundreds of thousands of questions whereas a search result with the Supabase tag generates hundreds of questions. The company is also backed by Google, which is one of the leading tech companies in the world. Therefore, the services it provides are required to meet the industry standards.

7.1.3 Firebase database

The biggest difference between Firebase and Supabase is the database they each offer. Supabase has a Postgres database while Firebase has a NoSQL

database. Firebase stores data inside documents containing key-value pairs whereas Supabase stores data inside tables with columns and rows.

NoSQL databases do not have a schema, as is the case with Firestore. Therefore, the database allows for flexibility since it is not a requirement to predefine the data structure. This lack of limitation can be especially good when dealing with unstructured data. Firestore also makes offline database persistence a possibility. With offline support, any local changes made while offline are automatically synchronized with the backend as soon as connection is established. Supabase does not offer this as of 2022.

7.1.4 Firebase functions

The type of function discussed here refers to serverless functions used to execute backend code. Firebase has Cloud Functions, which can be triggered by HTTPS requests or other Firebase features. For instance, background functions can be triggered based on events or just periodically.

An alternative to Cloud Functions in Supabase would be Edge Functions. Edge Functions are different from Cloud Functions in that the infrastructure built to run the function is geographically closer to end-users. Ultimately, the Edge Functions provided by Supabase operate similarly but there are a few limitations.

The first limitation is that only one function can be run in local development. The second limitation is that the function only supports POST responses, which means event-based triggers or scheduled tasks are not supported. Lastly, HTML responses are also not supported by Supabase Functions.

7.1.5 Firebase hosting

Firebase offers a service that handles hosting applications with either dynamic or static content. Most importantly, it does this securely with fast delivery [34].

Supabase on the other hand does not have a hosting service, which means another platform would be required for handling hosting.

Firebase also ensures that deployment can be done effortlessly by allowing developers to deploy their apps with a single command. Moreover, It ensures security as SSL is built directly into Firebase Hosting, and thus requires no configurations.

7.1.6 Firebase PWA features

Firebase offers several services that are more tailored to building PWAs. For instance, the Firebase Hosting serves applications over HTTPS with SSL certificates which adds safety and security. It can also cache dynamic and static content on a global CDN which can help improve load time. Moreover, Cloud Firestore supports offline data persistence which enables applications to fully operate while offline.

Firebase Authentication can also allow already authenticated users to remain signed in while offline by managing a local cache of sign-in data. Lastly, Firebase Cloud Messaging can improve user engagement by allowing users to send messages or push notifications to users based on certain events.

7.1.7 Firebase pricing

Firebase pricing is based on two tiers. The first tier is the Spark Plan which offers a lot of free features with a certain limitation. This tier is recommended for small projects that do not generate a lot of traffic. The second tier is the Blaze Plan or also known as the pay as you go plan. For this plan each individual service is offered as free but requires payment for any usage that surpasses the free threshold.

Table 2. Firebase price table for database, authentication, and storage services. [38]

Firestore Database	Free tier	Pay as you go tier
Authentication	Unlimited users	Unlimited users
Storage	5GB (storage) 1GB/day (transfer limit) 29k/day (upload operations) 50k/day (download operations)	\$0.18/GiB (network egress) \$0.18/100K (document writes) \$0.06/100K (document reads) \$0.02/100K (document deletes)

Table 2 demonstrates how large enterprise scale projects require more than the free tier to handle increased API calls. For small scale or personal projects, the free tier should be adequate. Firebase offers 1 GB of database storage for free which is a lot more than the 500MB of storage Supabase offers for the same tier. However, despite firebase offering larger database storage, it comes with limitations for reads, writes, and deletes made. If those limits are exceeded, then additional costs will be accrued.

The most notable difference between the free tier provided by Supabase and Firebase is the price for storage. Supabase offers 1 GB of storage for free and Firebase offers 5 times that. Moreover, the transfer limit offered by Supabase is 2 GB per month whereas Firebase offers a transfer limit of 1 GB per day. Therefore, for projects that are storage intensive, Firebase Storage might be ideal.

7.2 Advantages of Supabase over Firebase based on different aspects

7.2.1 Supabase features

Supabase has APIs that are auto generated based on the database, and this feature is exclusive to Supabase only. The APIs include auto-generated documentation which updates as changes are made to the database. This simplifies integrating Supabase to the client.

The syntax for handling errors is also simplified with Supabase, unlike Firebase where the syntax requires try and catch blocks to handle errors. For authentication, the developer experience offered is similar to Firebase, but Supabase has more third-party providers to select from. Additionally, Supabase has a great feature called Remote Procedure Call (RPC) which allows writing some logic to the database and then calling it directly from the application. This is useful when performing tasks where the logic does not change, such as account deletion, and password reset. It also has the benefit of having less client-side logic.

Lastly, Supabase offers Self Hosting where developers can deploy and personally manage the Supabase infrastructure. This is great because it gives developers the ability to personalize Supabase according to their needs. It also frees developers from vendor lock-in, so developers are not entirely dependent on the service provider, which is the case with Firebase.

7.2.2 Supabase database

Supabase has a Postgres database, which is a relational database. A relational database can be useful when dealing with tightly related data. Supabase is very useful for performing complex queries, joins, and full text searches. In fact, Firebase suggests using other services for handling text search since it does not support it.

The Firebase database struggles with this due to its document-based structure, which makes deeply nested references difficult to query. Another great database feature provided by Supabase is the ability to import directly from CSV or excel spreadsheet. This can be useful when migrating from a different platform. Additionally, as the database is updated, Supabase auto generates documentations used to handle CRUD operations more efficiently.

7.2.3 Supabase Pricing

Similar to Firebase, Supabase has a free tier but with limited amount of storage. Additionally, Supabase offers a pro tier with a base price of 28\$/month for production level application, and an enterprise tier for enterprise level projects.

Table 3. Supabase price table for database, authentication, and storage services. [39]

Supabase	Free tier	Pro tier
Database	Database space - Up to 500MB + unlimited API calls	Database space - 8 GB included, then \$0.125 per GB + Unlimited API calls
Authentication	10,000 users	Unlimited
Storage	Storage total - up to 1 GB Transfer limits - up to 2 GB	Storage total - 100GB included, then \$0.021 per GB Transfer limits - 200GB included, then \$0.09 per GB

Table 3 shows that Supabase offers unlimited API calls whereas Firebase charges for database reads and writes. If a project needs to make a lot of API calls, Supabase might be a great option in terms of pricing.

7.3 Performance Tests

The first performance measurement conducted was a speed test using the Lighthouse tool built by Google. In this instance, It is important to note that

version 8 and 9 of the Lighthouse Scoring Calculator was used for the test. The results acquired are estimated and vary at times so each website was analyzed a couple of times then the average result was calculated. The average overall performance score for the website built using Firebase was 65 out of 100, while the average overall performance score for the website built using Supabase was 76. The overall performance score is measured based on several metrics. Figure 25 shows the result of all the different metrics after a single analysis of each website.

The first metric looked at was the First Contentful Paint (FCP) metric which marks the time the first text or image is painted on the browser. The Firebase website had an average FCP of 1.6 seconds while the Supabase website had an average FCP of 1.7 seconds. As a reference, Google recommends an FCP of 0 to 18 seconds, so both websites produced a good result. The second metric looked at was the Time To Interactive (TTI) which measures the amount of time it takes for a page to become fully interactive. The Firebase website had an average TTI of 4.6 seconds while the Supabase website had an average TTI of 3.7 seconds. A TTI score in the range of 0 to 3.8 seconds is considered fast. Overall, the results of each metric combined indicated that the Supabase website had a faster speed performance.

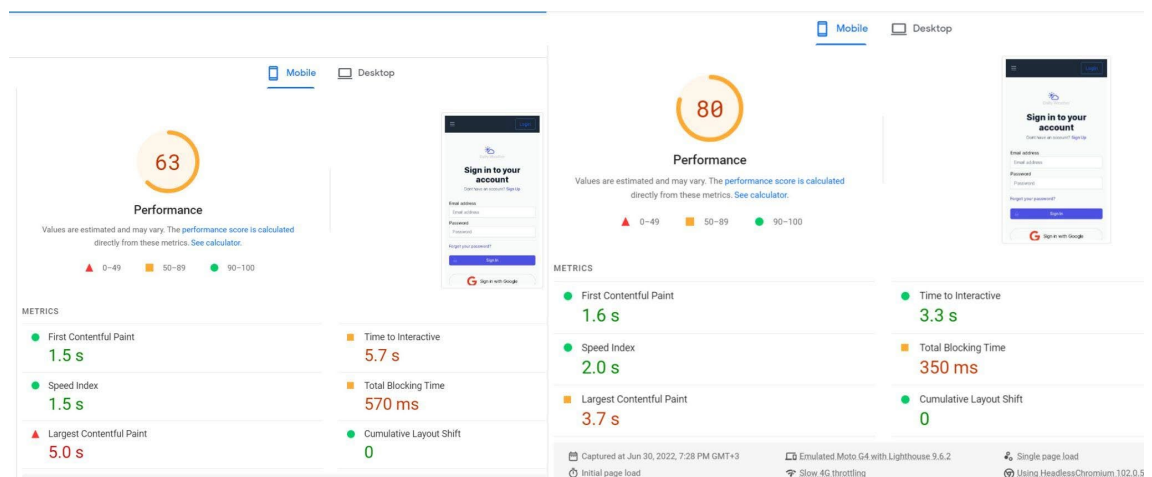


Figure 25. Speed test result of applications with Supabase on the right side and Firebase on the left side.

Load testing is a performance testing process where a software application is

examined under an expected load. It determines how the software will behave while being simultaneously accessed by a specific number of users. In this instance, Firestore and the Supabase database were tested to examine how many read and write operations can be made concurrently.

To perform the test, an open-source software called Apache JMeter¹⁰ was used. The test plan included 1000 threads, a ramp-up period of 1 second and a loop count of 1. A thread represents the number of users, and the ramp-up period defines how long it should take to get all the threads up and running. Lastly, the loop count determines how many time the test plan should be run. The first test was made by making a GET request to read a single document from the Firestore database. As seen in figure 26, after surpassing 965 requests, the test starts to fail.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
961	19:19:32.415	Users 1-278	HTTPS	7395	✓	1039	474	7391	7304
962	19:19:32.507	Users 1-370	HTTPS	7696	✓	1039	474	7696	7613
963	19:19:32.425	Users 1-288	HTTPS	7919	✓	1039	474	7917	7841
964	19:19:33.204	Users 1-858	HTTPS	15260	✓	1039	474	15260	15165
965	19:19:33.201	Users 1-880	HTTPS	15364	✓	1039	474	15364	15174
966	19:19:33.146	Users 1-844	HTTPS	21086	✗	2776	0	0	21086
967	19:19:33.203	Users 1-860	HTTPS	21029	✗	2776	0	0	21029
968	19:19:33.204	Users 1-908	HTTPS	21028	✗	2776	0	0	21028
969	19:19:33.205	Users 1-980	HTTPS	21027	✗	2776	0	0	21027
970	19:19:33.203	Users 1-882	HTTPS	21029	✗	2776	0	0	21029
971	19:19:33.202	Users 1-876	HTTPS	21031	✗	2776	0	0	21031
972	19:19:33.201	Users 1-869	HTTPS	21032	✗	2776	0	0	21032
973	19:19:33.205	Users 1-1000	HTTPS	21042	✗	2776	0	0	21042
974	19:19:33.226	Users 1-973	HTTPS	21022	✗	2776	0	0	21022
975	19:19:33.204	Users 1-997	HTTPS	21047	✗	2776	0	0	21047
976	19:19:33.222	Users 1-998	HTTPS	21039	✗	2776	0	0	21039
977	19:19:33.220	Users 1-998	HTTPS	21041	✗	2776	0	0	21041
978	19:19:33.228	Users 1-972	HTTPS	21034	✗	2776	0	0	21034
979	19:19:33.220	Users 1-999	HTTPS	21042	✗	2776	0	0	21042
980	19:19:33.221	Users 1-991	HTTPS	21041	✗	2776	0	0	21041

☐ Scroll automatically?
 ☐ Child samples?
 No of Samples: 1000
 Latest Sample: 21051
 Average: 4605
 Deviation: 3305

Figure 26. Load test result of Firestore database reads.

According to the Firebase documentation it is recommended to make at maximum 500 operations per second. Traffic can be increased incrementally every 5 minutes by 50% to give the database time to prepare for the growth in traffic. If done correctly, it is possible to make up to 740k operations per seconds after 90 minutes.

¹⁰ <https://jmeter.apache.org/>

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
943	20:37:05.875	Users 1-650	HTTPS	6088	🟢	1011	747	6088	5970
944	20:37:05.877	Users 1-654	HTTPS	6090	🟢	1011	747	6090	5984
945	20:37:05.897	Users 1-703	HTTPS	6074	🟢	1011	747	6070	5974
946	20:37:05.895	Users 1-671	HTTPS	6076	🟢	1011	747	6075	5971
947	20:37:05.915	Users 1-680	HTTPS	6062	🟢	1011	747	6060	5944
948	20:37:05.890	Users 1-602	HTTPS	6088	🟢	1011	747	6088	5964
949	20:37:05.902	Users 1-677	HTTPS	6076	🟢	1011	747	6076	5971
950	20:37:05.919	Users 1-726	HTTPS	6059	🟢	1011	747	6059	5951
951	20:37:05.884	Users 1-659	HTTPS	6095	🟢	1011	747	6095	5970
952	20:37:05.961	Users 1-734	HTTPS	6022	🟢	1011	747	6022	5898
953	20:37:05.916	Users 1-691	HTTPS	6072	🟢	1011	747	6072	5948
954	20:37:05.902	Users 1-709	HTTPS	6086	🟢	1011	747	6086	5968
955	20:37:05.887	Users 1-663	HTTPS	6101	🟢	1011	747	6101	5981
956	20:37:05.549	Users 1-338	HTTPS	6444	🟢	1011	747	6444	1321
957	20:37:05.526	Users 1-262	HTTPS	6575	🟢	1011	747	6575	1345
958	20:37:05.992	Users 1-765	HTTPS	7227	🟢	1011	747	7227	7114
959	20:37:05.981	Users 1-752	HTTPS	7238	🟢	1011	747	7238	7126
960	20:37:05.982	Users 1-754	HTTPS	21037	🔴	2776	0	0	21037
961	20:37:05.988	Users 1-761	HTTPS	21031	🔴	2776	0	0	21031
962	20:37:05.987	Users 1-760	HTTPS	21032	🔴	2776	0	0	21032

☐ Scroll automatically?
 ☐ Child samples?
 No. of Samples: 1000
 Latest Sample: 21028
 Success: 1246
 Deviation: 1709

Figure 27. Load test result of Firestore database writes.

The second load test was a Post request to create new documents. The thread properties of the test included 1000 threads, a ramp-up period of 1 second and a loop count of 1. As displayed in figure 27, the test plan resulted in an error after 959 threads, and any subsequent request was timed out.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1760	21:32:09.294	Thread Group 1-1216	HTTPS Reques	41575	🟢	1611	609	41575	41415
1761	21:32:09.612	Thread Group 1-1548	HTTPS Reques	41300	🟢	1611	609	41300	37381
1762	21:32:09.791	Thread Group 1-1764	HTTPS Reques	41230	🟢	1611	609	41223	41123
1763	21:32:09.812	Thread Group 1-1783	HTTPS Reques	41255	🟢	1611	609	41244	41147
1764	21:32:09.812	Thread Group 1-1785	HTTPS Reques	41264	🟢	1611	609	41264	41157
1765	21:32:09.794	Thread Group 1-1765	HTTPS Reques	41286	🟢	1611	609	41283	41185
1766	21:32:09.779	Thread Group 1-1750	HTTPS Reques	41312	🟢	1611	609	41312	41196
1767	21:32:09.675	Thread Group 1-1637	HTTPS Reques	41420	🟢	1611	609	41419	41320
1768	21:32:09.774	Thread Group 1-1745	HTTPS Reques	41337	🟢	1611	609	41336	41236
1769	21:32:09.710	Thread Group 1-1683	HTTPS Reques	41425	🟢	1611	609	41424	41306
1770	21:32:09.857	Thread Group 1-1830	HTTPS Reques	41288	🟢	1611	609	41288	41153
1771	21:32:09.101	Thread Group 1-1055	HTTPS Reques	42057	🔴	2839	0	0	42057
1772	21:32:09.639	Thread Group 1-1580	HTTPS Reques	41550	🟢	1611	609	41550	41419
1773	21:32:09.641	Thread Group 1-1606	HTTPS Reques	41548	🟢	1611	609	41548	41409
1774	21:32:09.790	Thread Group 1-1763	HTTPS Reques	41406	🟢	1611	609	41406	41293
1775	21:32:09.631	Thread Group 1-1591	HTTPS Reques	41575	🟢	1611	609	41575	41462
1776	21:32:09.731	Thread Group 1-1703	HTTPS Reques	41487	🟢	1611	609	41487	41382
1777	21:32:09.638	Thread Group 1-1562	HTTPS Reques	41581	🟢	1611	609	41581	41455
1778	21:32:09.680	Thread Group 1-1648	HTTPS Reques	41594	🟢	1611	609	41582	41461
1779	21:32:09.782	Thread Group 1-1752	HTTPS Reques	41495	🟢	1611	609	41495	41390

☐ Scroll automatically?
 ☐ Child samples?
 No. of Samples: 2000
 Latest Sample: 42101
 Success: 1767
 Deviation: 17076

Figure 28. Load test result of Supabase database reads.

Similarly, a load test was also conducted with the Supabase database, but both write and read operations had successfully passed the test. To further examine the limits of the database, the number of threads was increased to 2000 and a GET request was made to read the profile information of a specific user. As seen in figure 28, the first error occurred after 1770 threads and subsequent requests would sporadically result in failure. Lastly, a POST request was also made with similar thread properties which resulted in failure after 1184 threads.

8 Conclusion

The objective of this thesis was to evaluate two Backend-as-a-Service platforms, Firebase and Supabase. In doing so, the aim was to provide the case company, WeDance, with a collection of information that could be useful when deciding between the two platforms. Mainly, the evaluation was carried out based on performance, products offered, PWA services, stability, and pricing. The results were obtained by thoroughly reviewing existing writings on the topic as well as building a project that utilized tools from each platform.

The findings from the project revealed that determining the better platform is not as clear-cut as each platform has its own advantages and disadvantages. Selecting the right platform between the two depends on the use case of the project. For instance, Supabase offers a relational database which can be great for projects that require full-text search or need to make complex queries. Firebase on the other hand offers a hosting service as well as a NoSQL database which can be great for storing unstructured data.

The thesis addresses the criteria initially set and provides answers to questions asked in the research. Moreover, it describes the building process of a weather application that was successfully implemented using Firebase and Supabase. The work done in building the demo application had revealed which platform was more suitable for implementing certain functionalities. For instance, it was discovered that scheduled jobs are more difficult to implement with Supabase since Supabase does not offer a dedicated service for handling scheduled functions.

Ultimately, the results presented can be used to make an informed decision when making a choice between Firebase and Supabase. In the case of WeDance, one of the growing concerns with the current tool was the limitations faced when making complex queries therefore Supabase would be the recommendation for dealing with the concern. However, it is also recommended to look at other aspects to understand the trade off in switching between the two platforms.

Lastly, more research could be done by evaluating other aspects of Firebase and Supabase. The scalability of both databases could be further examined by simulating conditions in which both databases are put under excess load. Additionally, the self-hosting feature provided by Supabase could be tested by implementing a project where the infrastructure is self-managed.

References

- 1 WeDance. 2022. Online. <https://wedance.vip/about>. Accessed 20 Feb 2022.
- 2 Libguides.usc.edu. 2022. Research Guides: Organizing Your Social Sciences Research Paper: Types of Research Designs. Online. 9 May 2022. <https://libguides.usc.edu/writingguide/researchdesigns#:~:text=The%20research%20design%20refers%20to,measurement%2C%20and%20analysis%20of%20data>. Accessed 23 May 2022.
- 3 Pautasso, M. 2013. Ten Simple Rules for Writing a Literature Review. Online. 18 Jul 2013. <https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1003149>. Accessed 23 May 2022.
- 4 Streefkerk, R. 2022. Qualitative vs. quantitative research. Online. 7 Feb 2022 <https://www.scribbr.com/methodology/qualitative-quantitative-research/>. Accessed 23 May 2022.
- 5 Schwarzmüller M. 2022. Frontend vs Backend. Online. 2 July 2020. <https://academind.com/tutorials/frontend-vs-backend>. Accessed 20 Feb 2022.
- 6 Medium. 2022. The Fundamentals of Front End and Back End Development. Online. 31 Dec 2021. Sagara Technology Idea Lab. <https://sagaratechnology.medium.com/the-fundamentals-of-front-end-and-back-end-development-5973ac0910cf>. Accessed 23 Feb 2022.
- 7 Techterms.com. 2022. HTML (Hyper-Text Markup Language) Definition. Online. 23 May 2015. <https://techterms.com/definition/html>. Accessed 25 May 2022.
- 8 Developer.mozilla.org. 2022. JavaScript | MDN. Online. 27 Apr 2022. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. Accessed 11 May 2022.
- 9 Statista. 2022. Most used web frameworks among developers 2021 | Statista. Online. 18 Mar 2022. Lionel Sujay Vailshery.

- <https://www.statista.com/statistics/1124699/worldwide-developer-survey-most-used-frameworks-web>. Accessed 12 May 2022.
- 10 Codecademy. 2022. Back-End Web Architecture | Codecademy. Online. Codecademy Team <https://www.codecademy.com/article/back-end-architecture>. Accessed 23 Mar 2022.
 - 11 Techterms.com. 2022. Backend Definition. Online. 11 April 2020. <https://techterms.com/definition/backend>. Accessed 22 Mar 2022.
 - 12 Vercel. 2022. What is Next.js | Learn Next.js. Online. <https://nextjs.org/learn/foundations/about-nextjs/what-is-nextjs>. Accessed 23 May 2022.
 - 13 Reactjs.org. 2022. React – A JavaScript library for building user interfaces. Online. <https://reactjs.org/>. Accessed 23 May 2022.
 - 14 Tanna, Mayur, & Harmeet Singh. 2018. Serverless Web Applications with React and Firebase: Develop Real-Time Applications for Web and Mobile Platforms. Packt Publishing, Limited. ProQuest Ebook Central. Electronic Book. <https://ebookcentral.proquest.com/lib/metropolia-ebooks/reader.action?docID=5345875>. Accessed 23 May 2022.
 - 15 Cloudflare. 2022. What is BaaS. Online. <https://www.cloudflare.com/learning/serverless/glossary/backend-as-a-service-baas/>. Accessed 22 Mar 2022.
 - 16 Industryarc.com. 2022. Cloud/Mobile Backend as a Service Market Research Report: Market size, Industry outlook, Market Forecast, Demand Analysis, Market Share, Market Report 2020 - 2025. Online. <https://www.industryarc.com/Report/15097/cloud-mobile-backend-as-a-service-market.html#:~:text=The%20Cloud%2FMobile%20Backend%20as,22.9%25%20from%202021%20to%202026>. Accessed 23 Mar 2022.
 - 17 Batschinski. George. 2022. BaaS: What are the benefits? Online. 9 May 2022. <https://blog.back4app.com/what-are-the-benefits-baas-backend-as-a-service/>. Accessed 24 Mar 2022.
 - 18 Merixstudio.com. 2022. Backend as a Service (BaaS) vs. custom backend: Which one to choose and why. Online. 25 Nov 2019. <https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend/>. Accessed 23 Mar 2022.
 - 19 Geerligs, R., 2021. Software Developer Salary Around the World | 2021. Online. 17 Feb 2021. Daxx Software Development Teams. <https://www.daxx.com/blog/development-trends/it-salaries-software-developer-trends>. Accessed 24 Mar 2022.
 - 20 Developer.mozilla.org. 2022. Introduction to progressive web apps - Progressive web apps (PWAs) | MDN. Online. 1 May 2022.

- https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Introduction. Accessed 12 May 2022.
- 21 Richard, S. and LePage, P., 2022. What are Progressive Web Apps. Online. 24 Feb 2020. <https://web.dev/what-are-pwas/>. Accessed 12 May 2022.
 - 22 LePage, P. 2022. What does it take to be installable?. Online. 11 Jan 2022. <https://web.dev/install-criteria/>. Accessed 12 May 2022.
 - 23 Developer.mozilla.org. 2022. Service Worker API - Web APIs | MDN. Online. 20 May 2022. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API. Accessed 14 May 2022.
 - 24 Developer.mozilla.org. 2022. Cache - Web APIs | MDN. Online. 23 May 2022. <https://developer.mozilla.org/en-US/docs/Web/API/Cache>. Accessed 14 May 2022.
 - 25 W3schools.com. 2022. HTML Responsive Web Design. Online https://www.w3schools.com/html/html_responsive.asp. Accessed 15 May 2022. [25]
 - 26 Google Developers. 2022. How to Secure Your Site with HTTPS | Google Search Central | Documentation | Google Developers. Online. 21 Apr 2022. [https://developers.google.com/search/docs/advanced/security/https#:~:text=HTTPS%20\(Hypertext%20Transfer%20Protocol%20Secure,user%27s%20computer%20and%20the%20site](https://developers.google.com/search/docs/advanced/security/https#:~:text=HTTPS%20(Hypertext%20Transfer%20Protocol%20Secure,user%27s%20computer%20and%20the%20site). Accessed 15 May 2022.
 - 27 Firebase. 2022. Firebase Realtime Database | Firebase Documentation. Online. 18 May 2022. <https://firebase.google.com/docs/database#:~:text=each%20database%20instance,-.How%20does%20it%20work%3F,end%20user%20a%20responsive%20experience>. Accessed 23 May 2022.
 - 28 Dearmer, A., 2022. Firebase vs. MySQL: Battle of the Databases. Online. 1 Dec 2020. <https://www.integrate.io/blog/firebase-vs-mysql/>. Accessed 23 May 2022.
 - 29 Firebase. 2022. Cloud Firestore | Firebase Documentation. Online. 20 May 2022 <https://firebase.google.com/docs/firestore>. Accessed 21 May 2022.
 - 30 Firebase. 2022. Choose a Database: Cloud Firestore or Realtime Database | Firebase Documentation. Online. 18 May 2022. https://firebase.google.com/docs/database/rtdb-vs-firestore#key_considerations. Accessed 21 May 2022.

- 31 Firebase. 2022. Firebase Authentication | Firebase Documentation. Online. 18 May 2022. <https://firebase.google.com/docs/auth>. Accessed 22 May 2022.
- 32 Firebase. 2022. Firebase Storage | Firebase Documentation. Online. 18 May 2022. <https://firebase.google.com/docs/storage>. Accessed 22 May 2022.
- 33 Firebase. 2022. Firebase Functions | Firebase Documentation. Online. 18 May 2022. <https://firebase.google.com/docs/functions>. Accessed 22 May 2022.
- 34 Firebase. 2022. Firebase Hosting | Firebase Documentation. Online. 18 May 2022. <https://firebase.google.com/docs/hosting>. Accessed 22 May 2022.
- 35 Supabase.com. 2022. Introduction | Supabase. Online. <https://supabase.com/docs>. Accessed 23 May 2022.
- 36 Young, N., 2021. The Pros and Cons of Tailwind CSS | Webdesigner Depot. Online. 24 Sep 2021. Webdesigner Depot. <https://www.webdesignerdepot.com/2021/09/the-pros-and-cons-of-tailwind-css/>. Accessed 24 May 2022.
- 37 Google Developers. 2022. Lighthouse | Tools for Web Developers | Google Developers. Online. 13 Apr 2021. <https://developers.google.com/web/tools/lighthouse/>. Accessed 23 May 2022.
- 38 Firebase. 2022. Firebase Pricing | Firebase. Online. <https://firebase.google.com/pricing>. Accessed 23 May 2022.
- 39 Supabase. 2022. Pricing & fees | Supabase. Online. <https://supabase.com/pricing>. Accessed 23 May 2022.